

ShinkaEvolve: 迈向开放式与样本高效的程序进化

Robert Tjarko Lange, Yuki Imajuku 与 Edoardo Cetin Sakana AI

本文介绍 ShinkaEvolve¹：一个全新的开源框架，利用大语言模型（Large Language Models）以最先进的性能和前所未有的效率推动科学发现。近期在扩展大语言模型推理时间计算方面的进展，已使通用科学发现取得显著进步。这些方法依赖于进化式智能体框架，将大语言模型作为变异算子来生成候选解决方案。然而，当前的代码进化方法存在关键局限性：样本效率低下，需要数千个样本才能找到有效解决方案；且保持闭源，阻碍了广泛采用与扩展。ShinkaEvolve 针对这些局限性，引入三项关键创新：一种平衡探索与利用的父代采样技术、用于高效搜索空间探索的代码新颖性拒绝采样，以及基于多臂老虎机的 LLM 集成选择策略。我们在多种任务上评估 ShinkaEvolve，展示其在样本效率和解决方案质量上的一致改进。ShinkaEvolve 仅用 150 个样本就发现了新的最先进的圆形填充解决方案，为 AIME 数学推理任务设计了高性能的智能体框架，识别出 ALE-Bench 竞赛编程解决方案的改进，并发现了新颖的专家混合负载均衡损失函数，揭示了优化策略空间。我们的结果表明，ShinkaEvolve 以卓越的样本效率实现了广泛的适用性。通过提供开源可访问性和成本效率，这项工作使开放式发现能够在各种计算问题中普及。

Code <https://github.com/SakanaAI/ShinkaEvolve>

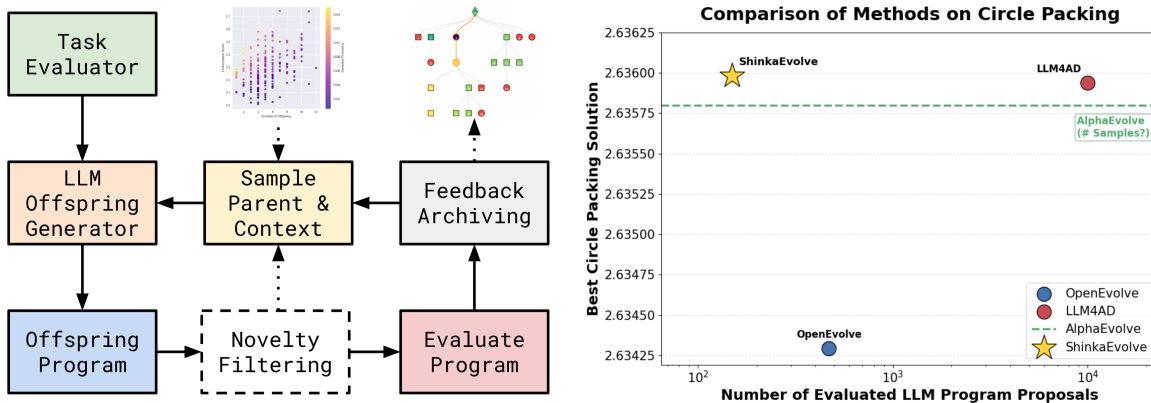


图 1 | ShinkaEvolve 的高层概览。左：ShinkaEvolve 框架构建已评估程序的档案，对新程序进行拒绝采样，并评估其适应度。右：ShinkaEvolve 提供了 AlphaEvolve 的样本高效替代方案，并超越了其圆形填充解决方案。

¹ 日语术语“shinka”意为“进化”或“创新”。ShinkaEvolve 指代一个开放式自我精炼创新引擎的愿景。虽然 ShinkaEvolve 对某些日语读者来说可能有点滑稽，因为它听起来像 Evolve-Evolve 或 进化-进化，但类似的汉语重复术语在日本相当常见。

1. 引言

大语言模型（Large Language Models, LLMs）的快速发展，通过能够自主开展实验并验证假设的智能体系统，推动了科学发现的变革（Lu 等，2024b；Novikov 等，2025；Yamada 等，2025；Zhang 等，2025）。这些框架将大语言模型用作复杂的变异算子，通过迭代优化候选解决方案，使成功变体在后续世代中得以传播。该方法论已在多个领域证明有效，例如竞赛编程（Li 等，2022）、数学优化（Romera-Paredes 等，2024）以及自动化智能体设计（Hu 等，2024）。然而，当前实现面临显著的实用局限性。首要挑战是严重的样本低效，现有方法通常需要数千次评估，导致计算成本高昂且耗时。这种低效源于朴素的探索策略，未能有效利用先前世代积累的知识。此外，大多数领先系统仍为闭源，这为可复现性设置了障碍，并限制了社区驱动的改进。ShinkaEvolve 通过三项关键的算法创新来应对这些挑战，这些创新协同作用以提升样本效率。我们的自适应父代与大语言模型采样，智能地平衡了对新颖区域的探索与对已知高质量区域的利用。其次，我们的代码提案新颖性拒绝采样确保了程序变异的高效性。最后，我们基于多臂老虎机的集成选择策略，能够动态适应采样存档父代与启发程序不断演变的状态。跨多个领域的实验验证表明，在效率和解决方案质量方面均有显著提升，ShinkaEvolve 以比现有方法少几个数量级的评估次数，达到了最先进的结果。通过将我们的完整实现作为开源软件发布，我们旨在使先进的进化发现工具更易于获取，并促进广泛的社区贡献。总结如下：

1. 本文提出 ShinkaEvolve，一个进化框架，通过三项关键算法创新显著提升样本效率：新颖的父程序采样策略、代码新颖性拒绝采样，以及基于性能的自适应大语言模型集成选择。
2. 本文在四个不同问题领域进行全面实验验证，展示 ShinkaEvolve 超越人类和大语言模型生成解决方案的创新能力：数学优化（圆形装箱）、智能体设计（AIME 任务）、竞赛编程（ALE-Bench）以及大语言模型训练设计（专家混合负载均衡损失）。
3. 本文以 Apache 2.0 许可证发布 ShinkaEvolve 开源软件，包括实现细节和用于监控搜索过程的交互式可视化工具。

2. 相关工作

基于大语言模型的进化代码优化 测试时计算的一种特定形式是进化代码优化：利用、变异和重组先前生成的代码以产生新样本。该方法此前已被用于优化奖励和偏好目标（Lu 等，2024a；Ma 等，2023）、数学科学代码（Romera-Paredes 等，2024）以及其他应用（Berman，2025；Lange 等，2024，2025；Lehman 等，2022；Meyerson 等，2023）。通过提示，大语言模型被用作重组引擎（Lange 等，2023；Meyerson 等，2023），能够模拟不同代码片段及其生成理由之间的交叉。这类程序档案构建系统类似于基于种群的、由大语言模型引导的树搜索（Inoue 等，2025；Jiang 等，2025）。与本文工作最密切相关的是 AlphaEvolve（Novikov 等，2025）、OpenEvolve（Sharma，2025）和 LLM4AD（Liu 等，2024a）。本文推进了这一研究方向，通过结合拒绝采样、大语言模型优先级排序和在线元草稿起草，展示了前所未有的样本效率。

开放式智能体发现 大语言模型与开放式进化原理的结合使智能体系统能够持续创新（Stanley 等，2017；Zhang 等，2025）。

与依赖显式多样性度量的传统新颖性搜索不同 (Lehman 和 Stanley, 2011; Lehman 等人, 2008), 大语言模型智能体利用学习到的表示来生成创造性解决方案, 同时保持语义连贯性 (Faldor 等人, 2024; Hu 等人, 2024; Novikov 等人, 2025)。这些智能体构建程序的进化树, 其中大语言模型引导的变异将跨代的关联解决方案连接起来 (Lehman 等人, 2020)。ShinkaEvolve 系统地结合了垫脚石——作为突破性创新构建的次优中间解决方案——通过使用大语言模型智能体来生成变异并评估程序关系, 使成功模式能够通过重组在搜索分支间迅速传播。

3. 方法

算法概述。 ShinkaEvolve 的控制流包含三个主要阶段:

1. 从岛屿程序子种群档案中采样父代和灵感。重要的是, 我们强调父代程序选择中探索与利用之间的权衡。
2. 通过大语言模型引导的代码编辑提议进行程序变异。我们利用基于代码嵌入相似度的新颖性拒绝采样, 以及大语言模型作为新颖性评判者的评估。
3. 程序执行和世界反馈引导大语言模型集成选择概率, 以及用于文档和知识传播的在线元草稿起草。

3.1. 父代和灵感采样 存档维护、岛屿种群与变异上下文构建。ShinkaEvolve 维护一个固定大小的存档, 其中包含先前评估过的程序及其适应度分数和元信息, 并实施精英规模约束。变异上下文包含一个主父代程序以及从表现最优的解决方案和随机存档样本中抽取的启发程序, 为大型语言模型提供多样化的示例样本以进行创造性重组。我们遵循 Novikov 等人 (2025) 和 Romera-Paredes 等人 (2024) 的方法, 采用岛屿模型, 其中独立的子种群由相同的初始程序播种。各岛屿并行进化以增强多样性并防止过早收敛。岛屿成员偶尔可以在岛屿之间迁移, 以在“发现子流”之间传播知识。为保护每个岛屿的独特性, 我们阻止岛屿特定的最佳表现程序迁移 (Romera-Paredes 等人, 2024; Tanese, 1989)。采样按层次进行: 首先从存档中均匀采样岛屿 ID, 随后将其用作父代和启发程序的来源。之后, 我们采样随机存档程序和表现前 K 的程序, 将它们用作上下文程序。

平衡探索与利用: 父代程序选择。 给定一个岛屿子种群, ShinkaEvolve 实现了多种不同的父代采样策略, 以平衡探索与利用: 首先, 我们采用幂律采样, 其中程序按适应度排序, 秩 r_i ($r_i = 1$ 表示最佳程序)。选择概率遵循 $p_i = \frac{r_i^{-\alpha}}{\sum_{j=1}^n r_j^{-\alpha}}$, 其中 α 控制利用强度。设置 $\alpha = 0$ 产生均匀采样, 而 $\alpha \rightarrow \infty$ 实现爬山法。受 Zhang 等人 (2025) 启发, 我们将其与加权采样进行对比, 后者结合了性能和 novelty。给定具有后代计数 $N(P_i)$ 的程序, 我们首先计算中位适应度 $\alpha_0 = \text{median}(\{F(P_1), F(P_2), \dots, F(P_n)\})$ 。性能分量使用 Sigmoid 缩放: $s_i = \sigma(\lambda \cdot (F(P_i) - \alpha_0))$ 其中 $\sigma(x) = \frac{1}{1 + e^{-x}}$ 且 λ 控制选择压力。novelty 分量 $h_i = \frac{1}{1 + N(P_i)}$ 偏向于后代较少的程序。最终概率结合这些因素: $p_i = \frac{w_i \cdot h_i}{\sum_{j=1}^n w_j \cdot h_j}$ 其中 $w_i = s_i$ 。平衡性能和 novelty。这些策略如图 2 所示。

3.2. 程序变异与新颖性评估

大型语言模型引导的程序变异。 为生成新程序, ShinkaEvolve 首先从

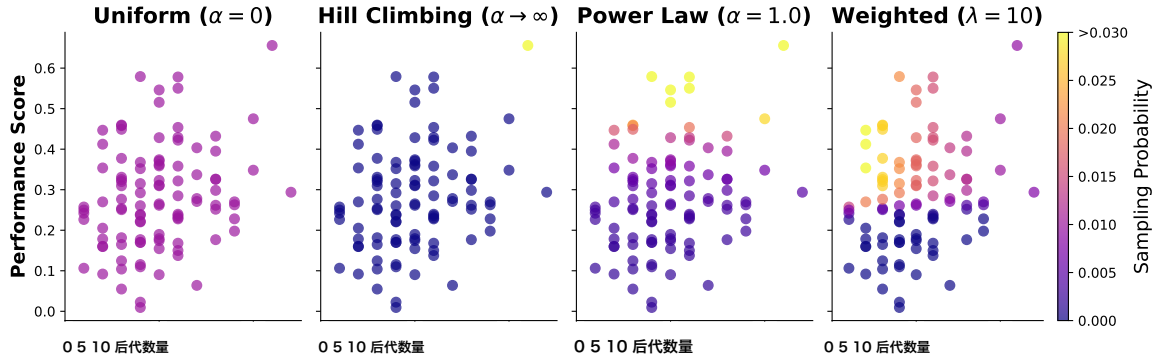


图 2 | ShinkaEvolve 父代采样。策略范围从纯探索（均匀采样）到纯利用（爬山法），再到性能与新颖性的结合。

预定义池。我们的框架支持来自领先应用程序编程接口提供商的模型，包括 GPT、Gemini、Claude 和 DeepSeek（Anthropic, 2024; Guo et al., 2025; OpenAI, 2023; Team, 2025）。在采样模型后，ShinkaEvolve 采用三种不同的变异方法来促进大语言模型生成程序变体的多样性和创造性：

1. **Diff-Based Edits.** We implement diff edits using LLMs following the approach outlined in
2. **完全重写。** 我们支持完全程序重写以提供更大的灵活性，并通过编程方式确保不可变块在大语言模型重写过程中保持不变。
3. **交叉变异。** 我们利用交叉变异（Lange et al., 2025; Lehman et al., 2022），其中采样一个额外的存档程序，并提示大语言模型组合程序。遵循 Novikov et al. (2025)，我们使用文本标记（EVOLVE-BLOCK-START 和 EVOLVE-BLOCK-END）确保不可变代码在大语言模型重写过程中保持不变。在获得代码更改提案后，我们强制不可变代码不被修改，如果补丁无效则重新采样新提案，并使用 Reflexion（Shinn et al., 2024）提供解析反馈。

通过新颖性拒绝采样实现程序多样性。 为了增强所执行代码提案的创造性，我们利用基础模型集成结合温度采样。此外，我们引入代码新颖性拒绝采样，使用嵌入模型对程序代码的可变部分进行嵌入。随后，我们计算岛屿子种群程序之间的余弦相似度分数。如果最大分数超过阈值（例如， $n = 0.95$ ），我们查询大语言模型以进一步评估该程序是否具有有意义的差异。该方法如图 3 所示。

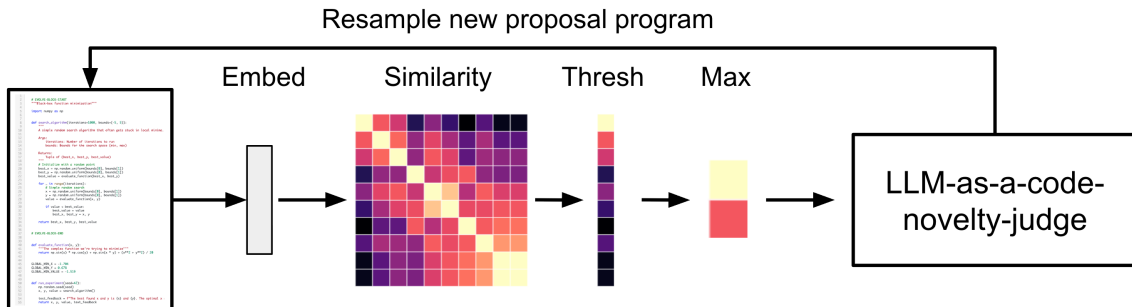


图 3 | ShinkaEvolve 程序新颖性拒绝采样。ShinkaEvolve 嵌入可变代码片段，计算档案中的相似度；如果最大分数超过阈值，则查询另一个大语言模型以评估该程序是否具有有意义的新颖性。

3.3. 执行与世界反馈

多目标优化与文本反馈。通过上述步骤获得的程序执行后，ShinkaEvolve 进行多目标评估，产生其标量适应度值 r_i 以及一组公开的“公共度量”和文本反馈。然后，ShinkaEvolve 存储这一完整的多目标评估在种群档案中，为未来几代语言模型突变提供信息丰富的上下文，使用简单的提示格式：

Example of Diff Edit Prompt with Textual Feedback

```
# Current program
Here is the current program we are trying to improve (you will need to propose a modification to it below):
```{language}
{code_content}
```

Here are the performance metrics of the program:
{performance_metrics}{text_feedback_section}

# Instructions
...
# Task
...
IMPORTANT: Do not rewrite the entire program - focus on targeted improvements.
```

自适应大语言模型采样进化。不同大语言模型（LLM）提出变异的性能会因问题领域以及当前采样档案父代与启发程序的状态而异。ShinkaEvolve 通过在每代结束时动态演化 LLM 采样概率来适应这种非平稳性。我们的方法基于 UCB1 算法（Auer 等人，2002），为每个 LLM 关联一个访问计数器和一个期望得分估计，并根据其采样变异的性能进行更新。我们引入了针对 LLM 驱动发现领域的定制修改。具体而言，我们不是使用每个变异的绝对适应度 r_i ，而是使用以下方式更新 LLM 分布：

$$r_i^u = \exp(\max(r_i - r_i^b, 0)) - 1, \text{ 其中 } r_i^b$$

是程序 i 的基线奖励，计算为其父代程序与数据库中初始程序之间的最大值，确保每个 LLM 根据其相对改进进行评估，以考虑程序档案的非平稳性。同时， $\exp(\cdot)$ 和 $\max(\cdot, 0)$ 操作有助于精确提升能够提出大胆、高风险、高回报变异的 LLM，而非“更安全”的微小改进。我们使用观察到的奖励的跟踪统计量来归一化 r_i^u ，并确保对每个领域的适应度尺度具有不变性。

元草稿与在线精化。ShinkaEvolve 实现了一个元草稿系统，定期分析成功解决方案以加速学习。每 T 代，我们总结最近的程序评估并识别常见的优化策略和设计原则。元代理将洞见综合为可操作的建议，附加到变异提示中，提供来自累积进化经验的高层指导。该方法如图 4 所示。

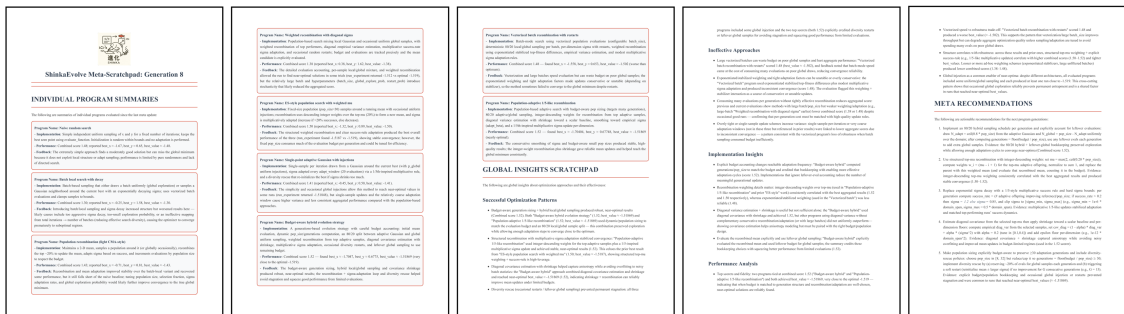


图 4 | ShinkaEvolve 元草稿。它由单个程序摘要、全局洞见和实现建议组成，这些内容被附加到变异提示中。

4. 结果

4.1. 圆形装箱：复现并改进 AlphaEvolve 结果

任务描述。圆形填充优化问题要求在单位正方形内放置 26 个圆，使得圆的半径之和最大化，同时确保圆之间不重叠且所有圆完全位于正方形边界内。这一约束优化挑战结合了离散放置决策与连续半径优化，使其成为进化算法的复杂基准。该问题存在多个局部最优解，需要复杂的搜索策略来发现高质量解，因为朴素方法往往收敛到空间利用率低的次优配置。

ShinkaEvolve 的发现动态。ShinkaEvolve 在 150 个进化世代中进行了评估，与通常需要数千次评估的现有方法相比，展示了显著的样本效率（图 1）。图 5（左）展示了改进轨迹，呈现出三个阶段：初始快速改进阶段，算法迅速发现基本的半径优化策略；持续探索阶段，随着更复杂技术（基于约束的优化）的出现而获得增量收益；最终收敛阶段，通过重启对最佳解进行细化。

图 5（右）揭示了成功创新如何在种群中传播，高性能解（以绿色和黄色显示）作为后续世代的父代。值得注意的是，该算法展示了复杂的探索模式，多个进化分支在收敛到以黑色突出显示的最优解路径之前探索了不同的算法方法。我们在第 5 节提供了各种消融研究。

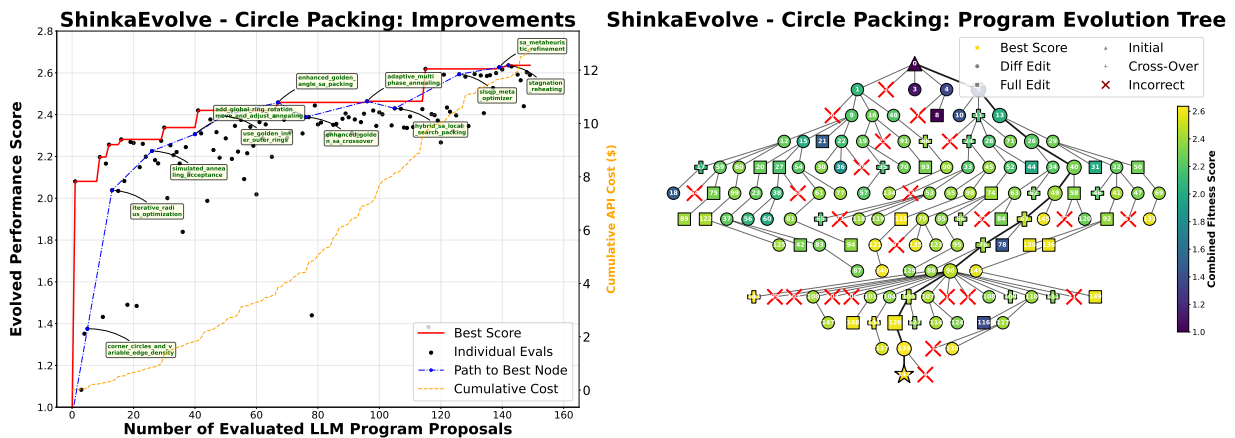


图 5 | ShinkaEvolve 在圆形填充任务上的表现。左：ShinkaEvolve 在不到 150 次程序评估内超越了 AlphaEvolve 的解。右：ShinkaEvolve 的程序进化树展示了将垫脚石迭代组合成高性能解的过程。

ShinkaEvolve 发现的解。演化得到的算法（第 C.1 节）融合了三项关键创新：（1）一种复杂的初始化策略，将圆以结构化的黄金角螺旋模式放置，并策略性地定位在角落和边缘；（2）一种混合优化方法，将 SLSQP 梯度优化与模拟退火相结合，用于全局探索；（3）智能扰动机制，交替进行局部圆移动和全局环境旋转，以跳出局部最优。所发现的解决方案采用自适应温度调度与再加热策略，以防止过早收敛，同时通过约束感知的半径计算保持可行性。这种从结构化初始化到元启发式探索再到基于梯度的精化的多层次方法，展示了 ShinkaEvolve 如何发现超越手工设计基线的复杂算法组合。

4.2. AIME：演化用于数学推理的智能体脚手架

任务描述。我们在 AIME 2024 (AIM, 2024) 数学推理问题上评估 ShinkaEvolve，该问题集包含 30 道具有挑战性的竞赛级题目，需要复杂的解题策略 (Hu 等, 2024)。任务涉及演化智能体脚手架设计，每个问题最多允许 10 次 LLM 查询，以保证计算效率。使用 gpt-4.1-nano 作为基础模型，我们进行了 75 代脚手架设计发现，每个候选方案在完整问题集上通过三次独立运行进行评估。

ShinkaEvolve 的发现动态。进化过程系统地探索提示策略、集成方法和验证技术，以识别最优智能体架构。ShinkaEvolve 发现的脚手架设计显著优于手工设计的基线，包括简单的单查询智能体和复杂的多数投票方法。搜索揭示了效率与性能之间的帕累托前沿 (图 6, 左)，其中 7 次大语言模型查询即可达到最大性能，而另一种脚手架使用完整的 10 次查询预算也能取得相当的结果。泛化实验揭示了关于脚手架鲁棒性的重要洞见。在 2023 年和 2025 年 AIME 问题上的评估显示出不同的迁移模式 (图 6, 中)：2023 年问题上较小的改进表明可能因训练数据污染而趋于饱和，而 2025 年问题上较大的提升则表明成功泛化到近期未见过的挑战。跨大语言模型迁移实验验证了鲁棒性，成功适配 gpt-4.1-mini、gpt-4.1 和 o4-mini，表明所发现的架构捕获了可泛化的策略，而非针对特定模型的优化 (图 6, 右)。

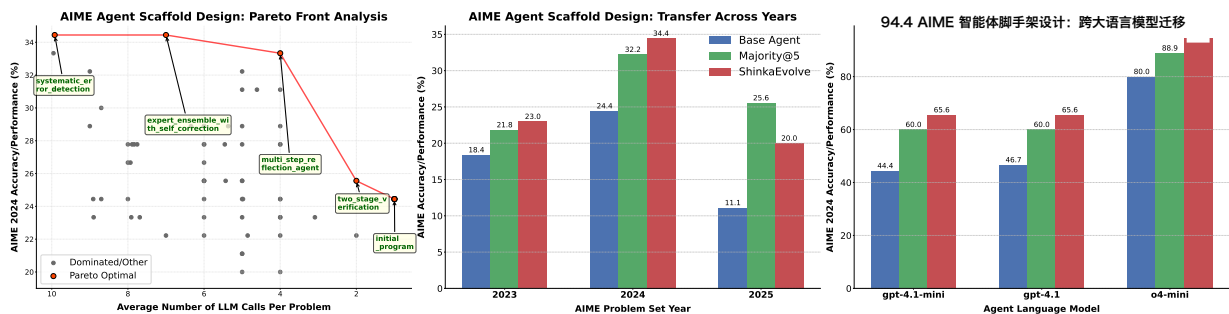


图 6 | ShinkaEvolve 用于智能体脚手架设计。左：ShinkaEvolve 发现性能与大语言模型查询预算之间的帕累托前沿。中：所发现的脚手架泛化到未见过的 AIME 问题。右：无论底层大语言模型如何，该脚手架都能提升性能。

ShinkaEvolve 发现的解。进化后的智能体采用三阶段架构，利用多样化的专家角色、批判性同行评审和综合机制。三位专业专家使用不同方法生成独立解决方案：一位细致的逐步推理者、一位直觉型模式识别专家和一位算法计算机科学导向的数学家，每位专家在 0.7 温度下运行，以平衡创造力与可靠性。第二阶段引入批判性同行评审，每个解决方案在低温 (0.1) 下接受一位持怀疑态度的审稿人的严格审查。审稿人通过在多个示例上测试模式来验证基于模式的推理，识别逻辑缺陷，并在必要时提供修正，从而显著提高解决方案质量。最后的综合阶段采用主编角色，在零温度下运行，分析所有解决方案和批评意见，确定最可靠的方法，并构建规范解决方案。稳健的回退机制在组件失败时，先求助于已评审解决方案的多数投票，再求助于原始解决方案，确保可靠输出。该架构在 10 次调用约束内有效利用了 7 次大语言模型调用 (3 次生成 + 3 次评审 + 1 次综合)。完整的已发现智能体脚手架见第 C.2 节。

4.3. ALE-Bench: 为组合优化进化程序

任务描述。我们将 ShinkaEvolve 应用于 ALE-Bench LITE (Imajuku 等人, 2025) 基准, 该基准包含 10 场由 AtCoder 主办的竞争性编程竞赛, 旨在测试大语言模型在启发式问题上的性能。在此, 我们探索 ShinkaEvolve 能否成功改进大语言模型发现的高性能解决方案。我们利用 ALE-Agent (Imajuku 等人, 2025) 发现的最佳编程解决方案作为每个问题的初始程序, 并应用 ShinkaEvolve 在其基础上进行改进。我们运行 ShinkaEvolve 50 代, 利用在公共测试集上计算的分数作为适应度函数。之后, 我们将最佳解决方案提交到私有测试集并报告分数。

ShinkaEvolve 的发现动态。ShinkaEvolve 能够将 ALE-Agent 在 10 个任务上发现的解平均提升约 2.3% (图 7)。此外, 在 ahc039 这一任务上, ShinkaEvolve 与 ALE-Agent 的组合若参与竞赛, 将在 AtCoder 排行榜上获得第二名。尽管这些提升源于细致的实现改进, 但我们观察到 ShinkaEvolve 提出的修改在算法上仍与 ALE-Agent 的初始化解保持接近。

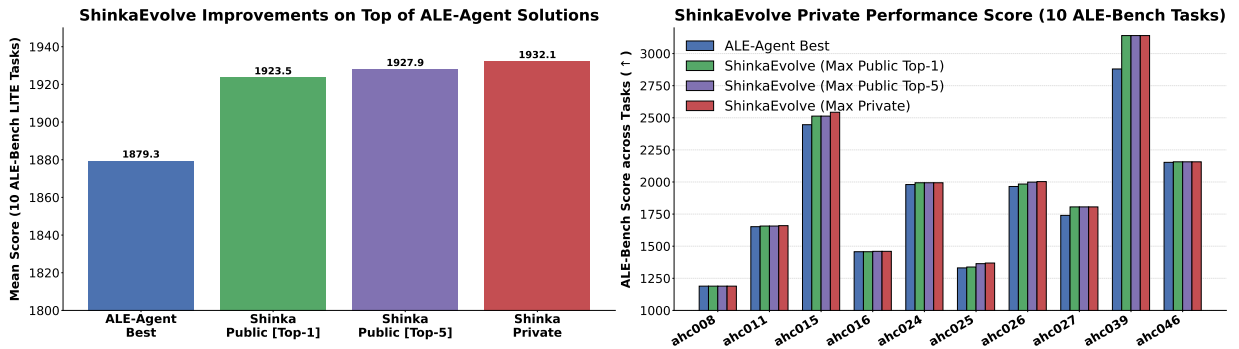


图 7 | ShinkaEvolve 用于改进 ALE-Bench 解。左: ShinkaEvolve 将 ALE-Agent 发现的解提升约 2.3%。右: 在 ahc039 任务上, 若参与竞赛, 解从 AtCoder 排行榜的第 5 名提升至第 2 名。

ShinkaEvolve 发现的解。我们聚焦于两个任务来说明 ShinkaEvolve 所发现的改进, 即 ahc039 和 ahc025。ahc039 的目标是寻找一个最优的、轴对齐的多边形, 在给定约束下最大化其包含的鲭鱼数量减去沙丁鱼数量。ALE-Agent 的基础解决方案采用模拟退火与 kd 树数据结构 (第 5 名, 性能 2880)。ShinkaEvolve 通过引入缓存验证过程和增强邻域算子等修改, 进一步改进了解决方案 (第 2 名, 性能 3140)。在缓存方面, kd 树被增强为在每个节点缓存子树统计信息, 包括包围盒和鱼类计数。在邻域算子方面, 引入了一种新颖的“定向边移动”, 它启发式地识别一个被错误分类的鱼 (例如, 多边形外的鲭鱼), 并贪婪地移动最近的边来纠正其状态。这些变化增强了搜索的方向性。对于 ahc025, 任务是使用天平比较任意两个物品子集的总重量, 目标是在固定次数的称量后, 将物品划分为总重量尽可能相等的组。ShinkaEvolve 通过引入更快的缓存、改进回退权重估计, 并最终用结合贪婪移动和定向局部搜索的更聚焦优化取代模拟退火, 改进了 ALE-Agent 的模拟退火基线。与顶尖人类解决方案的比较表明, 对于许多任务, 仍有很大的改进空间。此外, ShinkaEvolve 往往倾向于探索与 ALE-Agent 解决方案保持接近的修改。这表明存在对初始化解方案过拟合的潜在风险。

4.4. 大语言模型训练：为平衡且有效的专家演化损失函数

任务描述。专家混合（Mixture-of-Expert, MoE）架构（Fedus 等, 2022; Lepikhin 等, 2020; Shazeer 等, 2017; Szymanski 和 Lemmon, 1993）是一项关键进展，在现代开源与闭源旗舰模型中无处不在（Google AI Blog, 2024; Guo 等, 2025; Meta-AI, 2025; Team, 2025; Yang 等, 2025）。其基本思想很简单：用高效的小型模块（“专家”）集合取代传统的大型前馈残差块，每个专家可以专注于不同的问题领域（Fedus 等, 2022）。对于每个 MoE 层和每个词元，仅计算由路由器分类器选出的前 K 个专家的输出，从而有效拆分计算，使训练和推理更便宜、更快。然而，由于前 K 个专家选择操作不可微，必须为路由器提供辅助负载均衡损失（Load Balancing Loss, LBL），以避免词元负载过早坍塌为不均衡的专家分布。我们部署 ShinkaEvolve 正是为了应对这一开放的架构设计挑战，该挑战是推动近期 MoE 进展的核心焦点之一（Dai 等, 2024; Du 等, 2022; Fedus 等, 2022; Muennighoff 等, 2024; Qiu 等, 2025; Shazeer 等, 2017; Xue 等, 2024; Zoph 等, 2022）：设计有效的负载均衡损失，以激励效率 and 专业化，同时不损害模型的表达能力。

ShinkaEvolve 的发现动态。我们通过预训练一个具有 5.56 亿参数的混合专家模型来为负载均衡损失设计问题奠定基础， $N_r =$ 共有 64 个专家，其中对于任意给定词元仅有 $K = 8$ 个处于激活状态。这使得每次前向传播中稀疏激活的参数仅为 8200 万，不包括词元嵌入。我们在来自 fineweb（Penedo 等人, 2024）的超过 20 亿词元上训练这个小型模型，通过最小化混合专家损失函数，将负载均衡损失以 $\lambda = 0.01$ 的权重加到模型的交叉熵损失上。每个程序的适应度函数随后衡量一个简单目标：最小化最终交叉熵与模型“负载不均衡”之和，其中负载不均衡通过混合专家专家之间词元分布与均匀分布的 L1 偏差来衡量。考虑到预训练的成本，我们仅运行 ShinkaEvolve 30 次迭代。我们通过训练一个更大的具有 27 亿参数的混合专家模型，在略少于 300 亿的 fineweb 词元上，使用三个负载均衡系数 $\lambda \in 0.001, 0.01, 0.1$ ，产生不同级别的正则化，来评估 ShinkaEvolve 最佳解决方案的通用性。然后，我们与用于训练一些最流行开源大语言模型的“全局批量负载均衡损失”（Yang 等人, 2025）进行比较，比较最终困惑度（图 8，左）和下游任务性能（图 8，中），这些性能在七个不同的基准上评估（Bisk 等人, 2020; Clark 等人, 2018; Mihaylov 等人, 2018; Sakaguchi 等人, 2021; Sap 等人, 2019; Talmor 等人, 2018; Zellers 等人, 2019）。我们在下面以负载不均衡的函数形式提供结果，表明 ShinkaEvolve 的新损失在我们的训练配置中取得了一致的优势，并且随着 λ 系数的值增大而增大。

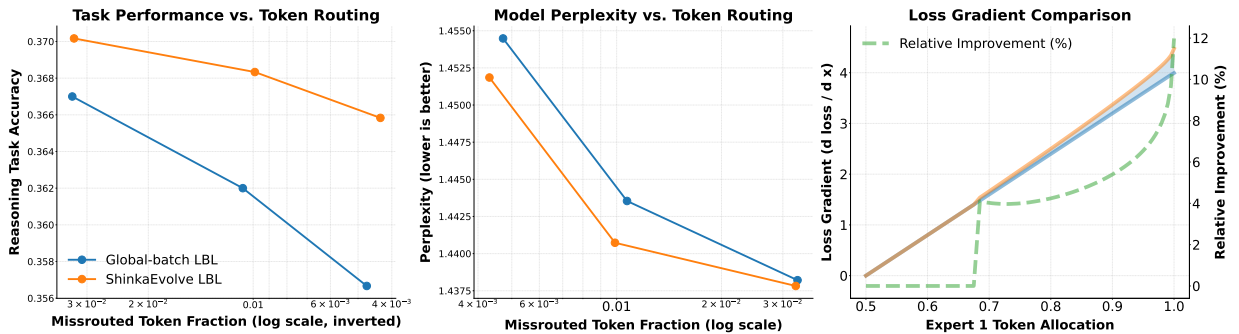


图 8 | ShinkaEvolve 用于发现混合专家负载均衡损失函数。左：七个基准上的下游任务性能。中：不同误路由分数下的最终困惑度。右：负载不均衡梯度作为词元分配的函数。

ShinkaEvolve 发现的解。所发现的负载均衡损失是对既有全局批量负载均衡损失的新改进，后者被用作进化搜索的种子。ShinkaEvolve 通过一个新项来补充这一流行的负载均衡损失，该新项专门针对正则化具有个别欠专业化专家的混合专家层。具体来说，令 $f_{\ell,i}$ 和 $P_{\ell,i}$ 对应于位于层 ℓ 的每个专家 i 的选择频率和平均路由概率。ShinkaEvolve 的负载均衡损失使用每层熵的归一化补集 $s(P_{\ell}) = 0.5 + 1 - \frac{H(P_{\ell})}{\log N_E}$

$$\tau = \frac{1}{N_E} \quad \text{and a minimum usage}$$

$$L_{\text{LBL}} = N_E \cdot \frac{1}{L} \sum_{\ell=1}^L \sum_{i=1}^{N_E} f_{\ell,i} P_{\ell,i} + \frac{0.1}{L} \sum_{\ell=1}^L s(P_{\ell}) \sum_{i=1}^{N_E} \max(0, \tau - f_{\ell,i}) . \quad (1)$$

ShinkaEvolve 新正则化 ShinkaEvolve 新正则化项的效果可以通过其在简化的双专家场景中对路由器令牌分配产生的诱导梯度来可视化（图 8，右）。直观上，该项会柔和地影响任意层的混合专家路由器，使专家被分配的令牌比例低于 τ 。乘子 $s(P_{\ell})$ 在层的路由熵 $H(P_{\ell})$ 较低且路由器集中于少数主导专家时，会增强这种推动力。这弥补了全局批次负载均衡损失的一个潜在盲点：即使少数专家几乎未被触及，点积 $f \cdot P$ 也可能看起来“均衡”。因此，ShinkaEvolve 的新项可视为一个安全网，一旦专家越过下限便自适应激活并消失，从而提供死专家并避免对已均衡层过度正则化。

5. 消融与分析

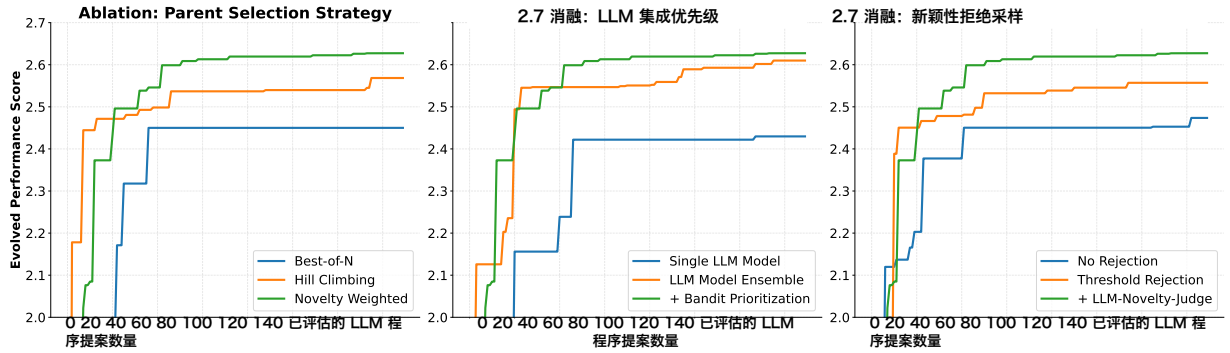


图 9 | ShinkaEvolve 方法在圆形填充上的消融研究。左：加权父代采样优于随机搜索和爬山法。中：基于老虎机的 LLM 集成相比固定均匀集成分布略微提升性能。右：以 LLM 作为新颖性评判的基于嵌入的拒绝采样显著优于无拒绝采样。

父代选择策略的影响。为了理解父代选择的重要性，我们比较了选择要进化程序的不同策略。Best-of-N 基线忽略进化历史，始终使用初始程序作为父代且无反馈。相比之下，爬山法代表一种贪心方法，始终选择性能最高的程序作为后续变异的父代。我们提出的加权采样策略通过基于适应度和后代数量概率性地选择父代，平衡了探索与利用。要点。加权采样在所有任务上一致优于随机搜索和爬山法。爬山法初期表现强劲但迅速趋于平台，而加权采样在整个进化过程中保持稳定改进。随机搜索收敛性最差，凸显了利用基于适应度的父代选择的重要性。

大语言模型集成与优先级排序的影响。进化智能体可以通过利用多个大语言模型（LLM）从多样化的编码能力中获益。我们通过将单一 LLM 基线（GPT-5-nano）与集成方法进行比较来研究这一假设。固定 LLM 集成通过从预定模型集合中均匀采样来提供多样性，而我们的基于多臂老虎机的 LLM 集成则自适应地学习哪些模型对适应度提升贡献最大，在探索未充分利用的模型与利用高性能模型之间取得平衡。要点。基于多臂老虎机的 LLM 集成显著优于单一 LLM 和固定集成方法。虽然固定集成相比单一 LLM 使用有适度改进，但自适应多臂老虎机策略通过根据模型对适应度提升的贡献动态优先选择更有效的模型，实现了最高性能。

基于代码嵌入的拒绝采样影响。相似的代码变体会浪费计算资源，而无法推进搜索前沿。为应对这一挑战，我们研究了不同的新颖性过滤机制。无拒绝采样基线接受任何 LLM 提议，可能允许近似重复的程序大量涌现。我们的基于嵌入的拒绝采样方法利用文本嵌入来识别并拒绝相似度得分超过 0.95 的提议。我们还探索了一种额外的 LLM 作为新颖性评判者的变体，该变体在基于嵌入的过滤基础上补充了 LLM 对程序新颖性的显式评估。要点。基于代码嵌入的拒绝采样通过防止冗余突变，相比无拒绝采样提供了显著的性能提升。额外的 LLM 作为新颖性评判者仅带来边际改进，这表明嵌入相似度已经是一种有效的新颖性评估代理，无需额外的计算开销。

6. 讨论

总结。本文介绍了 ShinkaEvolve，一个进化框架，通过提高样本效率和开源可访问性，解决了 LLM 驱动的科学发现中的关键局限性。ShinkaEvolve 在四个领域取得了最先进的结果：150 次评估的圆形装箱（数量级的改进）、复杂的 AIME 推理脚手架、ALE-Bench 算法改进以及新颖的专家混合负载均衡。

局限性。我们的实现采用固定配置，对探索与利用平衡的自动控制有限，这可能因领域而异。任务规范需要人工专业知识来定义目标函数和评估。该框架仅限于具有明确定义数值目标的问题，限制了其在多样化评估领域的适用性。

未来方向。通过大语言模型任务生成实现自动化任务规范，可以带来更大的自主性，并解锁未探索领域的应用。向真正的开放式演进过渡，即系统生成自身目标，代表了一个引人注目的前沿方向。自指优化和在线元学习为持续改进发现提供了机会。

更广泛影响与伦理考量。ShinkaEvolve 的开源发布进一步民主化了先进的进化优化，使之前无法获得专有系统的研究人员和从业者能够使用。该框架卓越的样本效率降低了资源受限环境下的计算障碍。然而，大规模使用大语言模型产生的应用程序编程接口成本可能造成经济障碍，潜在地制约民主化目标。

致谢

我们感谢 David Ha、Takuya Akiba、Taishi Nakamura、Luca Grilloti、Yutaro Yamada 以及 Sakana AI 团队的其他成员在整个项目中的有益讨论。

作者贡献列表

罗伯特·贾尔科·兰格 发起并领导了该项目，设计了核心的 ShinkaEvolve 代码库，并实现及收集了圆形填充、AIME 和 ALE-Bench 的结果。撰写了手稿。

今宿优希 协助搭建 ALE-Bench 基础设施，就 ALE-Bench 结果提供建议，并支持撰写手稿中关于 ALE-Bench 的部分。

埃多阿尔多·切廷 参与了 ShinkaEvolve 的设计讨论，提出并实现了自适应大语言模型采样方法及 Hydra 配置。他实现并收集了大语言模型训练和专家混合进化的结果。共同撰写了手稿。

参考文献

- American invitational mathematics examination, 2023. Problems and solutions, 2023. Published by the Mathematical Association of America / AMC contests.
- American invitational mathematics examination, 2024. Problems and solutions, 2024. Published by the Mathematical Association of America / AMC contests.
- American invitational mathematics examination, 2025. Problems and solutions, 2025. Published by the Mathematical Association of America / AMC contests.
- Anthropic. The claude 3 model family: Opus, sonnet, haiku, 2024. URL https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf.
- Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2):235–256, 2002.
- Jeremy Berman. How i got a record 53.6% on arc-agi. <https://jeremyberman.substack.com/p/how-i-got-a-record-536-on-arc-agi>, 2025. Accessed: 2025-02-08.
- Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. Glam: Efficient scaling of language models with mixture-of-experts. In *International conference on machine learning*, pages 5547–5569. PMLR, 2022.
- Maxence Faldor, Jenny Zhang, Antoine Cully, and Jeff Clune. Omni-epic: Open-endedness via models of human notions of interestingness with environments programmed in code, 2024. URL <https://arxiv.org/abs/2405.15568>.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Google AI Blog. Our next-generation model: Gemini 1.5. <https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024/>, February 2024. Accessed: 2025-07-13.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shitong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

- Yuki Imajuku, Kohki Horie, Yoichi Iwata, Kensho Aoki, Naohiro Takahashi, and Takuya Akiba. Ale-bench: A benchmark for long-horizon objective-driven algorithm engineering. *arXiv preprint arXiv:2506.09050*, 2025.
- Yuichi Inoue, Kou Misaki, Yuki Imajuku, So Kuroki, Taishi Nakamura, and Takuya Akiba. Wider or deeper? scaling llm inference-time compute with adaptive branching tree search. *arXiv preprint arXiv:2503.04412*, 2025.
- Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.
- Robert Lange, Tom Schaul, Yutian Chen, Tom Zahavy, Valentin Dalibard, Chris Lu, Satinder Singh, and Sebastian Flennerhag. Discovering evolution strategies via meta-black-box optimization. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pages 29–30, 2023.
- Robert Tjarko Lange, Yingtao Tian, and Yujin Tang. Large language models as evolution strategies. *arXiv preprint arXiv:2402.18381*, 2024.
- Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. The ai cuda engineer: Agentic cuda kernel discovery, optimization and composition. Technical report, Technical report, Sakana AI, 02 2025, 2025.
- Joel Lehman and Kenneth O Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary computation*, 19(2):189–223, 2011.
- Joel Lehman, Kenneth O Stanley, et al. Exploiting open-endedness to solve problems through the search for novelty. In *ALIFE*, pages 329–336, 2008.
- Joel Lehman, Jeff Clune, Dusan Misevic, Christoph Adami, Lee Altenberg, Julie Beaulieu, Peter J Bentley, Samuel Bernard, Guillaume Beslon, David M Bryson, et al. The surprising creativity of digital evolution: A collection of anecdotes from the evolutionary computation and artificial life research communities. *Artificial life*, 26(2):274–306, 2020.
- Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through large models, 2022. URL <https://arxiv.org/abs/2206.08896>.
- Dmitry Lepikhin, HyounJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.
- Fei Liu, Rui Zhang, Zhuoliang Xie, Rui Sun, Kai Li, Xi Lin, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Llm4ad: A platform for algorithm design with large language model. *arXiv preprint arXiv:2412.17287*, 2024a.

- Liyuan Liu, Young Jin Kim, Shuohang Wang, Chen Liang, Yelong Shen, Hao Cheng, Xiaodong Liu, Masahiro Tanaka, Xiaoxia Wu, Wenxiang Hu, et al. Grin: Gradient-informed moe. *arXiv preprint arXiv:2409.12136*, 2024b.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- Chris Lu, Samuel Holt, Claudio Fanconi, Alex James Chan, Jakob Nicolaus Foerster, Mihaela van der Schaar, and Robert Tjarko Lange. Discovering preference optimization algorithms with and for large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a. URL <https://openreview.net/forum?id=erjQDJ0z9L>.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024b.
- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023.
- Meta-AI. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation, 2025. URL <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- Elliot Meyerson, Mark J Nelson, Herbie Bradley, Adam Gaier, Arash Moradi, Amy K Hoover, and Joel Lehman. Language model crossover: Variation through few-shot prompting. *arXiv preprint arXiv:2302.12170*, 2023.
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*, 2018.
- Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, et al. Olmo: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.
- Alexander Novikov, Ng  n V  , Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- OpenAI. Gpt-4 technical report, 2023.
- Guilherme Penedo, Hynek Kydl  cek, Anton Lozhkov, Margaret Mitchell, Colin A Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.
- Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models. *arXiv preprint arXiv:2501.11873*, 2025.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995): 468–475, 2024.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

- Maarten Sap, Hannah Rashkin, Derek Chen, Ronan LeBras, and Yejin Choi. Socialliqa: Commonsense reasoning about social interactions. *arXiv preprint arXiv:1904.09728*, 2019.
- Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025. URL <https://github.com/codelion/openevolve>.
- Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.
- Kenneth O Stanley, Joel Lehman, and Lisa Soros. Open-endedness: The last grand challenge you’ve never heard of. *While open-endedness could be a force for discovering intelligence, it could also be a component of AI itself*, 2017.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomput.*, 568(C), February 2024. ISSN 0925-2312. doi: 10.1016/j.neucom.2023.127063. URL <https://doi.org/10.1016/j.neucom.2023.127063>.
- Peter T Szymanski and Michael D Lemmon. Adaptive mixtures of local experts are source coding solutions. In *IEEE International Conference on Neural Networks*, pages 1391–1396. IEEE, 1993.
- Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. Commonsenseqa: A question answering challenge targeting commonsense knowledge. *arXiv preprint arXiv:1811.00937*, 2018.
- Reiko Tanese. *Distributed genetic algorithms for function optimization*. University of Michigan, 1989.
- Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024.
- Gemini Team. Google deepmind. gemini 2.5: Pushing the frontier with advanced reasoning, multi-modality, long context, and next generation agentic capabilities. Technical report, Technical Report v2. 5, Google DeepMind, 2025.
- Fuzhao Xue, Zian Zheng, Yao Fu, Jinjie Ni, Zangwei Zheng, Wangchunshu Zhou, and Yang You. Open-moe: An early effort on open mixture-of-experts language models. *arXiv preprint arXiv:2402.01739*, 2024.
- Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. St-moe: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.

附录

A Shinka 实现细节 19

| | | | | | | | | |
|-------------|------------------|----|--------------------------------|----|------------------------|----|----------------------|----|
| B 任务实现细节 20 | B.1 圆形打包问题 | 20 | B.2 AIME 数学推理智能
体测试框架 | 22 | B.3 ALE-Bench 问题 | 23 | B.4 专家混合负载均衡损失 | 24 |
|-------------|------------------|----|--------------------------------|----|------------------------|----|----------------------|----|

| | | | | | | | | |
|---------------------------|------------------|----|-------------------------------|----|------------------------|----|----------------------|----|
| C ShinkaEvolve 发现的解决方案 28 | C.1 圆形打包问题 | 28 | C.2 AIME
数学推理智能体测试框架 | 31 | C.3 ALE-Bench 问题 | 33 | C.4 专家混合负载均衡损失 | 52 |
|---------------------------|------------------|----|-------------------------------|----|------------------------|----|----------------------|----|

A. Shinka 实现细节

- ShinkaEvolve 采用基于队列的实现，其中大语言模型（LLM）顺序生成程序提案。随后，这些提案被添加到作业评估队列中。每个提案都基于迄今为止已完成并存储在数据库中的所有作业。
- 在开发过程中，我们尝试了一种完全异步的实现，该实现同时利用作业队列和提案队列。这允许更高的吞吐量，但引入了一定程度的“非归档性”，即新的代码提案是提前生成的，并非基于所有先前提提交的作业。此外，来自查询速度更快模型的作业将被更早执行，因为它们的提案作业会被更早处理。关于吞吐量、样本效率和非归档性之间的最优权衡，仍存在许多开放的研究问题。
- 下面我们提供 Python 应用程序编程接口（API）的概述。它大致采用了 OpenEvolve (Sharma, 2025) 的高级接口：

```

从 shinka.core 导入 EvolutionRunner、EvolutionConfig；从 shinka.database 导入
DatabaseConfig；从 shinka.launch 导入 LocalJobConfig

# Minimal config - only specify what's required
job_config = LocalJobConfig(eval_program_path="evaluate.py")
db_config = DatabaseConfig()
evo_config = EvolutionConfig(init_program_path="initial.py",)

# Run evolution with defaults
runner = EvolutionRunner(
    evo_config=evo_config,
    job_config=job_config,
    db_config=db_config,
)
runner.run()

```

清单 1 | 最小化 ShinkaEvolve 配置与使用示例。

evaluate.py - 评估脚本

```

从 shinka.core 导入 run_shinka_eval

定义主函数 main(program_path: 字符串,
    results_dir: str):
    metrics, correct, err = run_shinka_eval(
        program_path=program_path,
        results_dir=results_dir,
        experiment_fn_name="run_experiment",
        num_runs=3, # Multi-vals to aggreg.
        get_experiment_kwargs=get_kwargs,
        aggregate_metrics_fn=aggregate_fn,
        validate_fn=validate_fn, # Optional
    )

def get_kwargs(run_idx: int) -> dict:
    return {"param1": "value", "param2": 42}

def aggregate_fn(results: list) -> dict:
    score = results[0]
    text = results[1]
    return {
        "combined_score": float(score),
        "public": {...}, # shinka-visible
        "private": {...}, # shinka-invisible
        "extra_data": {...}, # store as pkl
        "text_feedback": text, # str fb
    }

if __name__ == "__main__":
    # argparse program path & dir
    main(program_path, results_dir)

```

initial.py - 起始解决方案

```

# EVOLVE-BLOCK-START
def advanced_algo():
    # This will be evolved
    return solution
# EVOLVE-BLOCK-END

def run_experiment(**kwargs):
    """Main called by evaluator"""
    result = solve_problem(kwargs)
    return result

def solve_problem(params):
    solution = advanced_algo()
    return solution

```


B. 任务实现细节

B.1. 圆形填充问题

详细任务描述。圆形填充任务要求在单位正方形内放置 26 个圆，使得半径之和最大化，同时确保圆之间不重叠且所有圆完全位于正方形边界内。

带松弛的验证方法论。对于本文中展示的主要 ShinkaEvolve 运行，我们采用了 OpenEvolve (Sharma, 2025) 提供的验证脚本，该脚本允许 1×10^{-6} 的数值松弛。为确保结果的鲁棒性，我们额外使用 AlphaEvolve (Novikov 等人, 2025) 的精确验证代码对解进行了验证。我们发现，通过将每个圆的半径减少 1×10^{-8} ，所发现的解可以轻松变为精确解，这证明了我们方法的高精度。从松弛公式到精确公式的调整使所发现解的半径之和减少了可忽略不计的量，从 2.635983099011548 降至 2.6359828390115476，相对变化小于 10^{-6} 。

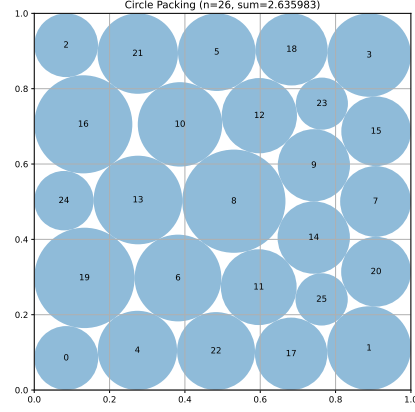


图 10 | ShinkaEvolve 发现的圆形填充解。

带精确约束的验证方法论。此外，我们使用 AlphaEvolve 的精确验证代码复现了该解，如图 11 所示，得分为 2.63597770931127。该解的发现需要评估更多样本。这阐明了一个重要原则：在进化过程中可以有效地使用替代的松弛任务，随后进行后处理以发现精确的最先进解。

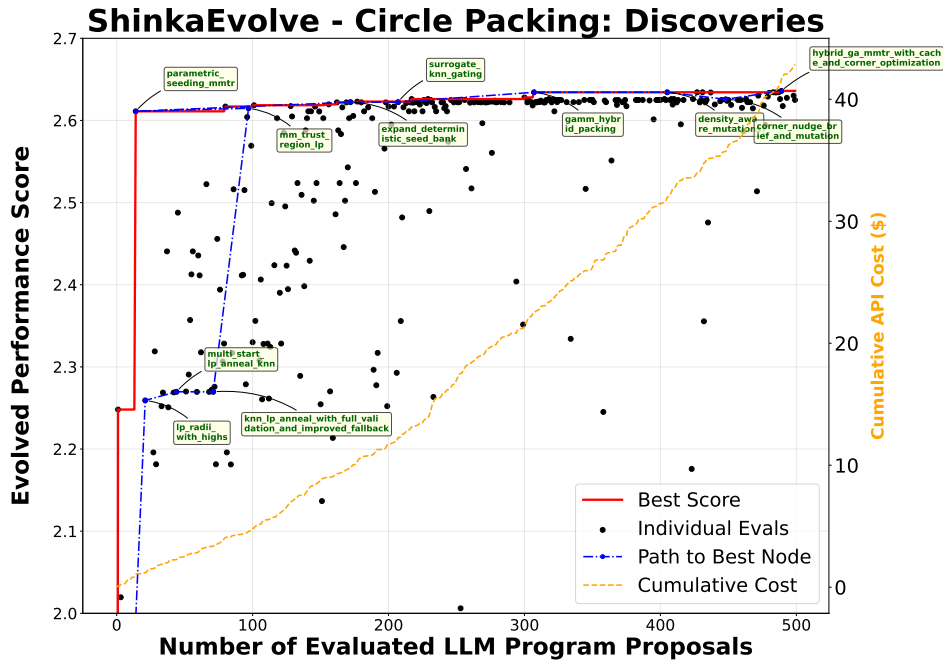


图 11 | 精确圆形填充验证的圆形填充异步进化结果，展示了收敛行为和解质量随时间的变化。

基线比较。我们的性能基准是针对三种解决方案建立的

主要来源。AlphaEvolve 的半径总和取自其论文（Novikov 等人，2025）。OpenEvolve 基线分数源自其官方实现及仓库中提供的示例。此外，本文与 LLM4AD 的结果进行比较，特别是其圆形装箱实现和启发式演化（EoH）实验配置。这些基线全面覆盖了现有的自动化算法设计方法，从而能够对本文方法进行公平且全面的性能评估。

SHINKAEVOLVE’s Hyperparameter Configuration.

| Parameter | Value | Parameter | Value |
|--------------------------------|------------------------|---------------------------|-------------------|
| Database configuration | | | |
| Archive size | 40 | Elite selection ratio | 0.3 |
| Archive inspirations | 4 | Top- k inspirations | 2 |
| Migration interval | 10 | Migration rate | 0.0 |
| Island elitism | true | Parent selection strategy | weighted |
| Parent selection λ | 10.0 | Number of islands | 2 |
| Evolution configuration | | | |
| Patch types | [diff, full, cross] | Patch type probs | [0.45, 0.45, 0.1] |
| Generations | 150 | Max parallel jobs | 5 |
| Max patch resamples | 3 | Max patch attempts | 3 |
| Meta recommendation interval | 10 | Max meta recommendations | 5 |
| Embedding model | text-embedding-3-small | Max novelty attempts | None |
| Code embed sim threshold | 0.95 | Problem implementation | Python |
| LLM dynamic selection | ucb1 | Exploration coefficient | 1.0 |
| LLM models | | | |
| gemini-2.5-pro | × | gemini-2.5-flash | × |
| claude-sonnet-4 | ✓ | o4-mini | ✓ |
| gpt-5 | × | gpt-4.1-nano | ✓ |
| gpt-4.1 | ✓ | gpt-4.1-mini | ✓ |
| LLM settings | | | |
| Temperatures | [0.0, 0.5, 1.0] | Max tokens | 16,384 |
| Meta models | [gpt-5-nano] | Meta temperatures | [0.0] |
| Novelty models | [gpt-5-nano] | Novelty temperatures | [0.0] |

表 1 | 圆形填充任务的 ShinkaEvolve 超参数配置。

B.2. AIME 数学推理智能体测试框架

详细任务描述。针对智能体脚手架设计任务，我们在 AIME 2024 数学推理问题上评估 ShinkaEvolve，该问题集包含 30 道具有挑战性的竞赛级题目，需要复杂的解题策略（AIM, 2024）。出于计算和成本效率的考虑，我们将每个问题的大语言模型查询次数限制为最多 10 次。使用 gpt-4.1-nano 作为基础模型，我们在 75 代内进化脚手架设计。此外，为了应对大语言模型查询中的随机性，我们在完整问题集上对每个候选方案进行了三次独立运行的评估。进化完成后，我们在 2023 年和 2025 年的 AIME 问题上评估所发现的脚手架设计（AIM, 2023, 2025），以评估其泛化能力以及对不同基础智能体语言模型的鲁棒性。

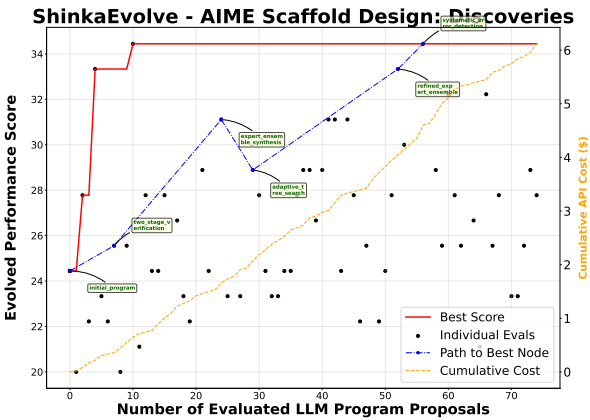


图 12 | ShinkaEvolve 在数学智能体脚手架设计中的发现轨迹。

SHINKAEVOLVE’s Hyperparameter Configuration.

| Parameter | Value | Parameter | Value |
|------------------------------|------------------------|---------------------------|-----------------|
| Database configuration | | | |
| Archive size | 40 | Elite selection ratio | 0.3 |
| Archive inspirations | 4 | Top-k inspirations | 2 |
| Migration interval | 10 | Migration rate | 0.1 |
| Island elitism | true | Parent selection strategy | weighted |
| Parent selection λ | 10.0 | Number of islands | 4 |
| Evolution configuration | | | |
| Patch types | [diff, full, cross] | Patch type probs | [0.6, 0.3, 0.1] |
| Generations | 75 | Max parallel jobs | 1 |
| Max patch resamples | 3 | Max patch attempts | 3 |
| Meta recommendation interval | 10 | Max meta recommendations | 5 |
| Embedding model | text-embedding-3-small | Max novelty attempts | 3 |
| Code embed sim threshold | 0.95 | Problem implementation | Python |
| LLM dynamic selection | null | Exploration coefficient | 0.0 |
| LLM models | | | |
| gemini-2.5-pro | ✓ | gemini-2.5-flash | × |
| claude-sonnet-4 | ✓ | o4-mini | ✓ |
| gpt-5 | × | gpt-5-nano | × |
| gpt-4.1 | × | gpt-4.1-mini | × |
| LLM settings | | | |
| Temperatures | [0.0, 0.5, 1.0] | Max tokens | 16,384 |
| Meta models | [gpt-4.1] | Meta temperatures | [0.0] |
| Novelty models | [gpt-4.1] | Novelty temperatures | [0.0] |

表 2 | 数学推理智能体框架（Math Reasoning Agentic Harness）的 ShinkaEvolve 超参数配置。

B.3. ALE-Bench 问题

详细任务描述。ALE-Bench 基准 (Imajuku 等, 2025) 是一组此前用于竞技编程竞赛 (AtCoder) 的启发式编程问题。我们在由 10 个问题组成的 LITE 子集上评估 ShinkaEvolve。我们遵循 ALE-Agent (Imajuku 等, 2025) 的评估协议, 并使用在公开测试用例上计算的分数作为适应度函数。随后, 我们将最佳解提交到私有测试集并报告分数。此外, 在图 7 中, 我们提供了对公开评分前 5 的解进行评估并取其在私有测试集上的最大分数的分数。虽然这不同于传统的竞技编程设置, 但它使我们能够评估所发现解的泛化能力。平均解分数从 1923.5 略微提高到 1927.0。因此, 我们没有观察到对公开测试用例过拟合的显著证据。

SHINKAEVOLVE’s Hyperparameter Configuration.

| Parameter | Value | Parameter | Value |
|--------------------------------|---------------------|---------------------------|-----------------|
| Database configuration | | | |
| Archive size | 50 | Elite selection ratio | 0.3 |
| Archive inspirations | 2 | Top-k inspirations | 2 |
| Migration interval | 10 | Migration rate | 0.1 |
| Island elitism | true | Parent selection strategy | weighted |
| Parent selection λ | 10.0 | Number of islands | 2 |
| Evolution configuration | | | |
| Patch types | [diff, full, cross] | Patch type probs | [0.6, 0.3, 0.1] |
| Generations | 50 | Max parallel jobs | 1 |
| Max patch resamples | 3 | Max patch attempts | 3 |
| Meta recommendation interval | 5 | Max meta recommendations | 5 |
| Embedding model | None | Max novelty attempts | None |
| Code embed sim threshold | None | Problem implementation | C++ |
| LLM dynamic selection | ucb1 | Exploration coefficient | 1.0 |
| LLM models | | | |
| gemini-2.5-pro | ✓ | gemini-2.5-flash | ✓ |
| claude-sonnet-4 | ✓ | o4-mini | ✓ |
| gpt-5 | ✓ | gpt-5-mini | ✓ |
| gpt-4.1 | × | gpt-4.1-mini | × |
| LLM settings | | | |
| Temperatures | [0.0, 0.5, 1.0] | Max tokens | 16,384 |
| Meta models | [gpt-5-mini] | Meta temperatures | [0.0] |
| Novelty models | None | Novelty temperatures | None |

表 3 | ALE-Bench 问题的 ShinkaEvolve 超参数配置。

B.4. 专家混合负载均衡损失

| Hyperparameter | Small MoE (evolution) | Large MoE (evaluation) |
|--|----------------------------------|----------------------------------|
| Model architecture | | |
| Model parameters | 556M | 2.7B |
| Model parameters | 82M | 404M |
| Number of experts (N_E) / active per token (K) | 64 / 8 | 64 / 8 |
| Hidden size | 512 | 1024 |
| Hidden size in each MoE expert | 384 | 768 |
| Number of hidden layers | 12 | 16 |
| Number of attention heads | 8 | 16 |
| Number of key-value heads | 8 | 8 |
| Head dimension | 128 | 128 |
| Attention bias | false | false |
| Attention dropout | 0.0 | 0.0 |
| Initializer range | 0.02 | 0.02 |
| RoPE θ | 1,000,000 | 1,000,000 |
| Tied word embeddings | true | true |
| Output router logits | true | true |
| Decoder sparse step | 1 | 1 |
| Router auxiliary loss coefficient (λ) | 0.01 | 0.001, 0.01, 0.1 |
| Computation dtype | bfloat16 | bfloat16 |
| Training setup | | |
| Optimizer | AdamW | AdamW |
| Learning rate | 1.0×10^{-3} | 3.0×10^{-4} |
| Weight decay | 0.1 | 0.1 |
| Adam parameters ($\beta_1, \beta_2, \epsilon$) | (0.9, 0.95, 1×10^{-8}) | (0.9, 0.95, 1×10^{-8}) |
| Learning rate scheduler | Cosine decay | Cosine decay |
| Warmup steps | 70 | 490 |
| Maximum sequence length | 1024 | 1024 |
| Global train batch size (sequences) | 1024 | 2048 |
| Tokens per training step | 1,048,576 | 2,097,152 |
| Maximum steps | 2000 | 14,000 |
| Total tokens | 2.10B | 29.36B |
| Dataset | fineweb | fineweb |

表 4 | MoE 架构与训练设置。

详细任务描述。专家混合 (Mixture-of-Expert, MoE) 架构 (Fedus 等, 2022; Lepikhin 等, 2020; Shazeer 等, 2017; Szymanski 和 Lemmon, 1993) 是一项关键进展, 使大语言模型训练的规模扩展取得突破。MoE 目前在现代开源和闭源旗舰模型中无处不在 (Google AI Blog, 2024; Guo 等, 2025; Meta-AI, 2025; Team, 2025; Yang 等, 2025)。MoE 设计的核心原理是用较小模块 (“专家”) 的集合取代传统的大型前馈残差块, 这些模块可以在训练期间高效分片, 并在推理期间仅部分激活 (Fedus 等, 2022)。每个专家本身是一个小型前馈网络 $E_{\ell,i}$, 位于第 ℓ 层大小为 N_F 的更大集合中。路由器是一个层特定的线性分类器 h_ℓ , 为每个标记选择最相关的 top- K 个专家, 仅计算它们的输出:

$$y_\ell(x) = \sum_{i=1}^{N_E} g_{\ell,i}(x) E_{\ell,i}(x), \quad g_{\ell,i}(x) = \begin{cases} \frac{e^{h_{\ell,i}(x)}}{\sum_{j \in \mathcal{T}_K(x)} e^{h_{\ell,j}(x)}}, & \text{if } i \in \mathcal{T}_K(x) \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

其中 $\mathcal{T}_K(x)$ 表示对应于前 K 个路由器 logits $h_{\ell,i}(x)$ 的索引集合。这种稀疏激活设计允许不同专家专注于不同的问题领域，从而在处理多样化提示时实现更高的效率、可扩展性和适应性。然而，由于前 K 个专家选择操作不可微，为路由器提供辅助负载均衡损失（LBL）至关重要。LBL 可防止令牌分布不均和专家专业化不足的崩溃。设计一种有效的负载均衡损失，在不妨碍表达性的同时鼓励效率和专家专业化，仍然是一个开放的设计挑战，推动了 MoE 领域近期的大部分进展（Dai 等，2024；Du 等，2022；Fedus 等，2022；Muennighoff 等，2024；Qiu 等，2025；Shazeer 等，2017；Xue 等，2024；Zoph 等，2022）。微小的设计变化已被证明会显著影响效率 and 专业化能力（Dai 等，2024；Jiang 等，2024；Liu 等，2024b；Qiu 等，2025；Team，2024）。最广泛采用的设计之一是 Shazeer 等（2017）提出的“全局批次”LBL，它支撑了多个最先进的开放模型，如 Qwen 3（Yang 等，2025）。对于具有 N_E 个专家的层 ℓ ，其定义为：

$$L_{LB} = N_E \cdot \frac{1}{L} \sum_{\ell=1}^L \sum_{i=1}^{N_E} f_{\ell,i} \cdot P_{\ell,i}, \quad (3)$$

其中

$f_{\ell,i}$ = 路由到专家 i 的令牌数 / 层中总令牌数 $\bar{\ell}$ ，该公式鼓励各专家的令牌使用量与路由器的平均软分配概率保持一致。
 $P_{\ell,i} = \frac{\sum_x h_{\ell,i}(x)}{\sum_{x,j} h_{\ell,j}(x)}$

我们通过预训练一个具有 556M 参数的 MoE 模型来评估 ShinkaEvolve， $N_E = 64$ 个专家中每个令牌仅激活 $K = 8$ 个，对应于每次前向传播中 82M 稀疏激活参数（不包括嵌入）。训练在来自 fineweb（Penedo 等，2024）的 2B 令牌上进行。对于每个程序，我们定义一个适应度函数，由交叉熵（CE）损失和加权为 $\lambda = 0.01$ 的 LBL 项组成。为了额外衡量负载不平衡，我们跟踪令牌分配与均匀分布的 L1 偏差：

$$L_{imb} = \frac{1}{2} \sum_{i=1}^{N_E} \left| f_{\ell,i} - \frac{1}{N_E} \right|, \quad (4)$$

值越低表示负载分布越均匀。这一基础为 ShinkaEvolve 提供了双重搜索目标：最小化 CE 同时改善负载平衡。为避免局部噪声影响交叉熵计算，我们在最后 10M 令牌上取平均。进化过程中使用的最终适应度分数为两者之和的负值：

$$r = -(L_{CE} + L_{imb}). \quad (5)$$

鉴于预训练成本高昂，我们仅对 ShinkaEvolve 进行 30 次迭代，重点关注 gpt-4.1、gemini-2.5-pro 和 claude-sonnet-4。为评估通用性，我们扩展到一个更大的 27 亿参数 MoE，其中 4.04 亿活跃参数（不含嵌入），在略低于 300 亿 fineweb 标记上训练，并比较三个 LBL 系数 $\lambda \in \{0.001, 0.01, 0.1\}$ 。我们使用 AdamW（Loshchilov 和 Hutter，2017）作为优化器，采用余弦衰减和线性预热。与现代训练惯例一致，我们使用旋转位置嵌入（Su 等，2024）、SwiGLU MLP（Shazeer，2020）和半精度 bfloat16 以高效地将模型权重保留在设备上。对于 ShinkaEvolve 进化过程中使用的小模型，我们使用略高于 1M 个标记的批大小，

共 2K 步。对于较大的 MoE，批大小加倍，总步数增加七倍。训练后，我们以困惑度（图 8，左）和七个标准评估的下游性能为基准，与全局批 LBL 基线进行比较：CommonSense QA（Talmor 等，2018）、HellaSwag（Zellers 等，2019）、OpenBook QA（Mihaylov 等，2018）、PIQA（Bisk 等，2020）、SIQA（Sap 等，2019）、WinoGrande（Sakaguchi 等，2021）和 ARC（Clark 等，2018），对于大型基准，将问题数量截断为 1000，如（Penedo 等，2024）所做。

如第 4 节所述并在附录 C 中详细说明，ShinkaEvolve 发现了对公式 3 中全局批 LBL 的一种新改进，该公式用于初始化进化搜索。ShinkaEvolve 发现通过添加一个额外的正则化项来增强该损失，以针对欠专业化的专家。如公式 3 所定义，令 $f_{\ell,i}$ 和 $P_{\ell,i}$ 分别表示第 ℓ 层中专家 i 的选择频率和平均路由概率。此外，定义 $s(P_{\ell}) = 0.5 + 1 - \frac{H(P_{\ell})}{\log N_E}$

作为路由熵的归一化补集，并令 $\tau = 0.064/N_E$ 为最小使用阈值。最终发现的 LBL 为：

$$L_{\text{LBL}} = \underbrace{N_E \cdot \frac{1}{L} \sum_{\ell=1}^L \sum_{i=1}^{N_E} f_{\ell,i} P_{\ell,i}}_{\text{全局批量负载均衡损失}} + \underbrace{\frac{0.1}{L} \sum_{\ell=1}^L s(P_{\ell}) \sum_{i=1}^{N_E} \max(0, \tau - f_{\ell,i})}_{\text{ShinkaEvolve 新正则化}} . \quad (6)$$

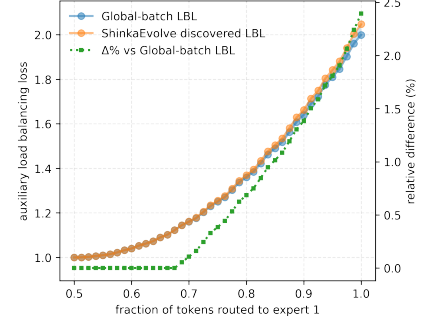


图 13 | LBL 损失比较。

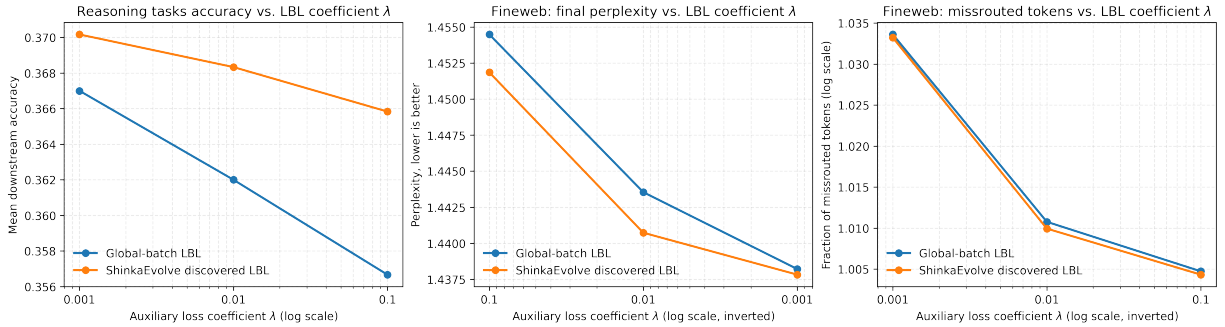


图 14 | 专家混合 LBL 设计的附加结果。

除第 4 节的结果外，图 14 还提供了全局批量负载均衡损失（LBL）与 ShinkaEvolve 发现的 LBL 的额外对比结果。具体而言，我们报告了平均任务性能、最终困惑度以及误路由令牌的比例，作为用于训练专家混合模型（MoE）的 LBL 系数 λ 的函数。与此前的分析一致，ShinkaEvolve 的 LBL 似乎在两个维度上均优于原始 LBL。然而，我们也注意到，用于进化和测试所采用 LBL 的架构非常相似，且训练预算仍然有限。不过，在不同训练预算和系数 λ 下的一致泛化结果为未来扩展到更长的训练周期提供了乐观的前景，届时即使微小的效率提升也可能转化为显著的成本节约。

SHINKAEVOLVE's Hyperparameter Configuration.

| Parameter | Value | Parameter | Value |
|--------------------------------|------------------------|---------------------------|------------|
| Database configuration | | | |
| Archive size | 20 | Elite selection ratio | 0.3 |
| Archive inspirations | 4 | Top- k inspirations | 2 |
| Migration interval | 10 | Migration rate | 0.1 |
| Island elitism | true | Parent selection strategy | weighted |
| Parent selection λ | 10.0 | Number of islands | 2 |
| Evolution configuration | | | |
| Patch types | [diff, full] | Patch type probs | [0.5, 0.5] |
| Generations | 20 | Max parallel jobs | 1 |
| Max patch resamples | 10 | Max patch attempts | 10 |
| Meta recommendation interval | 10 | Max meta recommendations | 5 |
| Embedding model | text-embedding-3-small | Max novelty attempts | 3 |
| Code embed sim threshold | 0.95 | Problem implementation | Python |
| LLM dynamic selection | ucb1 | Exploration coefficient | 1.0 |
| LLM models | | | |
| gemini-2.5-pro | ✓ | gemini-2.5-flash | × |
| claude-sonnet-4 | ✓ | o4-mini | × |
| gpt-5 | × | gpt-5-nano | × |
| gpt-4.1 | ✓ | gpt-4.1-mini | × |
| LLM settings | | | |
| Temperatures | [0.0, 0.5, 1.0] | Max tokens | 16,384 |
| Meta models | [gpt-4.1] | Meta temperatures | [0.0] |
| Novelty models | [gpt-4.1] | Novelty temperatures | [0.0] |

表 5 | ShinkaEvolve 用于 MoE LBL 发现的超参数配置。

C. ShinkaEvolve 发现的解决方案

C.1. 圆填充问题

```
# 进化 - 块 - 开始 导入 numpy 为 np; 从 scipy.optimize 导入
minimize, Bounds

def construct_packing():
    """
    Constructs an arrangement of 26 circles by combining a meta-heuristic
    search with a powerful SLSQP optimizer for refinement.
    """
    n = 26

    # --- Helper functions for the optimizer ---
    def objective_func(x):
        """The function to be minimized: the negative sum of radii."""
        return -np.sum(x[:n])

    def constraints_func(x):
        """
        Computes constraint violations. For SLSQP, each value must be >= 0.
        """
        radii = x[:n]
        centers = x[n:].reshape((n, 2))

        containment = np.concatenate(
            [
                centers[:, 0] - radii,
                centers[:, 1] - radii,
                1 - centers[:, 0] - radii,
                1 - centers[:, 1] - radii,
            ]
        )

        overlap = []
        for i in range(n):
            for j in range(i + 1, n):
                dist = np.linalg.norm(centers[i] - centers[j])
                overlap.append(dist - (radii[i] + radii[j]))

        return np.concatenate([containment, np.array(overlap)])

    def _compute_initial_radii(centers):
        """
        Computes a valid set of initial radii for a given set of centers
        to create a feasible starting point (x0) for the optimizer.
        """
        radii = np.min(
            [centers[:, 0], centers[:, 1], 1 - centers[:, 0], 1 - centers[:, 1]], 轴=0)

        对 _在范围内 (100):
            improved = False
            for i in range(n):
                for j in range(i + 1, n):
                    dist = np.linalg.norm(centers[i] - centers[j])
                    if radii[i] + radii[j] > dist:
                        excess = (radii[i] + radii[j] - dist) * 0.501
                        total_r = radii[i] + radii[j]
                        if total_r > 1e-9:
                            radii[i] -= excess * (radii[i] / total_r)
                            radii[j] -= excess * (radii[j] / total_r)
                            improved = True
            if not improved:
                break
        return np.maximum(radii, 1e-6)

    # --- 1. Generate a single high-quality initial guess ---
    centers_init = np.zeros((n, 2))
    inset = 0.06
    centers_init[0:4] = [
        [inset, inset],
        [1 - inset, inset],
        [inset, 1 - inset],
        [1 - inset, 1 - inset],
    ]
    centers_init[4:8] = [[0.5, inset], [0.5, 1 - inset], [inset, 0.5], [1 - inset, 0.5]]
    centers_init[8] = [0.5, 0.5]

    golden_angle = np.pi * (3 - np.sqrt(5))
    cx, cy = 0.5, 0.5
    inner_r, outer_r = 0.23, 0.48
    inner_idx, outer_idx = np.arange(9, 15), np.arange(15, 26)

    for i, idx in enumerate(inner_idx):
        angle = i * golden_angle
        centers_init[idx] = [cx + inner_r * np.cos(angle), cy + inner_r * np.sin(angle)]
    for i, idx in enumerate(outer_idx):
        angle = i * golden_angle * 1.003
        centers_init[idx] = [cx + outer_r * np.cos(angle), cy + outer_r * np.sin(angle)]

    centers_init += np.random.uniform(
```

```

-0.01, 0.01, size=(n, 2)
) # Increased initial jitter
centers_init = np.clip(centers_init, 0.01, 0.99)

# --- 2. Define bounds and constraints for the solver ---
bounds = Bounds([0.0] * n + [0.0] * (2 * n), [0.5] * n + [1.0] * (2 * n))
constraints = {"type": "ineq", "fun": constraints_func}

# --- 3. Initial baseline optimization ---
radii_init = _compute_initial_radii(centers_init)
x0 = np.concatenate([radii_init, centers_init.flatten()])

结果 = 最小化(
    objective_func,
    x0,
    method="SLSQP",
    bounds=bounds,
    constraints=constraints,
    options={"maxiter": 600, "ftol": 1e-8, "disp": False},
) # Increased initial maxiter

# Initialize current and best solutions for SA
best_x = result.x.copy()
current_x = result.x.copy()
best_score = -result.fun
current_score = -result.fun

# --- 4. Simulated Annealing loop: Perturb and refine with acceptance criterion ---
sa_iterations = 250 # Significantly increased iterations for SA
temperature = 0.05 # Initial temperature for SA
initial_temperature = temperature # Preserve for potential reheating
cooling_rate = 0.995 # Slower cooling rate for broader search
perturb_step = 0.04 # Initial step size for perturbations
initial_perturb_step = perturb_step # Preserve for potential reheating
step_decay = 0.999 # Decay rate for step size
last_improve = 0 # Iteration of last best improvement
stagnation_limit = sa_iterations // 4 # Iterations before triggering reheating

for iter_idx in range(sa_iterations):
    candidate_centers = (
        current_x[n:].reshape((n, 2)).copy()
    ) # Start from current state

    # Select a move type: 70% local, 30% global ring rotation
    if np.random.rand() < 0.7:
        # Local move: perturb a few circles
        num_to_move = np.random.randint(2, 6)
        indices = np.random.choice(n, num_to_move, replace=False)
        candidate_centers[indices] += np.random.normal(
            0, perturb_step, size=(num_to_move, 2)
        )
    else:
        # Global move: rotate one of the rings
        idx_to_rotate = inner_idx if np.random.rand() < 0.5 else outer_idx
        center_point = candidate_centers[8] # Center of the overall pattern
        angle = np.random.normal(
            0, 0.15
        )
        # Angular perturbation (can be fixed or scaled)
        rel_pos = candidate_centers[idx_to_rotate] - center_point
        cos_a, sin_a = np.cos(angle), np.sin(angle)
        rotated = np.column_stack(
            [
                cos_a * rel_pos[:, 0] - sin_a * rel_pos[:, 1],
                sin_a * rel_pos[:, 0] + cos_a * rel_pos[:, 1],
            ]
        )
        candidate_centers[idx_to_rotate] = center_point + rotated

    candidate_centers = np.clip(candidate_centers, 0.01,
0.99) # 裁剪以保持边界内

    # Create a new starting point and run a shorter refinement optimization
    x0_candidate = np.concatenate(
        [_compute_initial_radii(candidate_centers), candidate_centers.flatten()]
    )
    refine_result = minimize(
        objective_func,
        x0_candidate,
        method="SLSQP",
        bounds=bounds,
        constraints=constraints,
        options={"maxiter": 150, "ftol": 1e-6, "disp": False},
    ) # Reduced maxiter, looser ftol

    new_score = -refine_result.fun

    # Simulated Annealing Acceptance Criterion
    # Accept if better, or with probability if worse (based on temperature)
    if new_score > current_score or (
        temperature > 1e-7
        and np.random.rand() < np.exp((new_score - current_score) / temperature)
    ):
        current_score = new_score
        current_x = refine_result.x.copy() # Update current state

```

```

        if new_score > best_score:
            best_score = new_score
            best_x = refine_result.x.copy() # Update global best
            last_improve = iter_idx # Reset stagnation counter on improvement
# If not accepted, current_x remains unchanged for the next iteration (implicit)

# Cool down temperature and decay perturbation step size
temperature *= cooling_rate
perturb_step *= step_decay
if temperature < 1e-7:
    temperature = 1e-7 # Prevent division by zero
if perturb_step < 1e-5:
    perturb_step = 1e-5 # Prevent step from becoming too small
# Reheat if stagnated beyond stagnation_limit
if iter_idx - last_improve > stagnation_limit:
    temperature = initial_temperature
    perturb_step = initial_perturb_step
    last_improve = iter_idx

# --- 5. Final Polishing Run on the best found solution ---
final_result = minimize(
    objective_func,
    best_x,
    method="SLSQP",
    bounds=bounds,
    constraints=constraints,
    options={"maxiter": 1000, "ftol": 1e-9, "disp": False},
) # Increased maxiter for final polish

# Check if the final polishing improved the best_x from SA
if -final_result.fun > best_score:
    best_x = final_result.x.copy() # Make sure to copy

# --- 6. Unpack and return the best result ---
final_radii = best_x[:n]
final_centers = best_x[n:].reshape((n, 2))
return final_centers, final_radii

def compute_max_radii(centers):
    """
    This function is retained for structural compatibility with the evaluation
    framework but is not used by the new `construct_packing` logic.
    It computes maximum radii for a fixed set of centers.
    """
    n = centers.shape[0]
    radii = np.empty(n)
    for i in range(n):
        x, y = centers[i]
        radii[i] = min(x, y, 1 - x, 1 - y)

    对 _在范围内 (60):
        improved = False
        for i in range(n):
            for j in range(i + 1, n):
                dist = np.linalg.norm(centers[i] - centers[j])
                if radii[i] + radii[j] > dist:
                    excess = (radii[i] + radii[j] - dist) * 0.5
                    total = radii[i] + radii[j]
                    if total > 0:
                        reduce_i = excess * (radii[i] / total)
                        reduce_j = excess * (radii[j] / total)
                        radii[i] = max(0.001, radii[i] - reduce_i)
                        radii[j] = max(0.001, radii[j] - reduce_j)
                    improved = True
            if not improved:
                break
    return radii

# 进化-块-结束

# This part remains fixed (not evolved)
def run_packing():
    """Run the circle packing constructor for n=26"""
    np.random.seed(7)
    centers, radii = construct_packing()
    # Calculate the sum of radii
    sum_radii = np.sum(radii)
    return centers, radii, sum_radii

圆心、半径、sum_radii = run_packing()

```

清单 2 | ShinkaEvolve 发现的圆形填充方案。

C.2. AIME 数学推理智能体测试框架

"""针对数学任务的智能体设计评估。"""

导入 re; 从 typing 导入 Callable、List、Optional、Tuple、Dict; 从 collections 导入 Counter、defaultdict; 从 math_eval 导入 agent_evaluation

```
# EVOLVE-BLOCK-START
import re
from collections import Counter

class Agent:
    def __init__(
        self,
        query_llm: Callable,
        temperature=0.0,
    ):
        self.query_llm = query_llm
        self.output_format_instructions = "On the final line output only the digits of the answer (0-999). Provide your final answer enclosed in a LaTeX \\boxed{...} command."

        # Parameters
        self.generation_temperature = 0.7
        self.review_temperature = 0.1
        self.synthesis_temperature = 0.0

        # Use 3 experts to stay within a 10-call limit (3 gen + 3 review + 1 synth = 7 calls)
        self.num_experts = 3
        self.expert_personas = [
            "You are a meticulous and cautious mathematician. Your guiding principle is 'slow and steady wins the race'. You solve problems by breaking them down into the smallest possible steps based on fundamental principles. You avoid leaps of logic and verify each step before proceeding.",
            "You are a brilliant and intuitive mathematician, known for finding elegant, non-obvious solutions. You look for symmetries, invariants, or a change of perspective that radically simplifies the problem. You trust your insights but explain them clearly.",
            "You are a mathematician with a strong background in computer science. You approach problems by trying to frame them algorithmically. You think in terms of states, transitions, and recurrence relations, and you analyze the behavior of these systems to find the solution.",
        ]

    def _extract_answer(self, text: str) -> Optional[str]:
        """Extracts the final answer from a \\boxed{} environment."""
        if not text:
            return None
        matches = re.findall(r"\\boxed{\\d{1,3}}", text)
        if matches:
            return matches[-1]
        return None

    def forward(self, problem: str) -> tuple[str, float]:
        """
        Solves a problem using a multi-persona ensemble with peer review and synthesis.
        """
        total_cost = 0.0

        # === STAGE 1: Generate Diverse Solutions with Expert Personas ===
        solutions = []
        for i in range(self.num_experts):
            persona = self.expert_personas[i % len(self.expert_personas)]
            prompt = f"Solve the following AIME problem by thinking step-by-step. {self.output_format_instructions}\\n\\nPROBLEM:\\n{problem}\\n\\nSOLUTION:"
            try:
                响应、成本 = self.query_llm(
                    prompt=prompt,
                    system=persona,
                    temperature=self.generation_temperature,
                )
                solutions.append(response)
                total_cost += cost
            except Exception:
                # If a query fails, we proceed with fewer solutions.
                solutions.append(f"Expert {i + 1} failed to generate a solution.")

        # === STAGE 2: Independent Peer Review & Self-Correction ===
        critiques = []
        reviewer_system_prompt = "You are a skeptical peer reviewer examining a proposed solution to an AIME problem. Your task is to be extremely critical. Do not accept any statement at face value. Re-read the original problem carefully. Check calculations. Scrutinize the logical flow. **Pattern Verification:** If the solution relies on a pattern, you MUST test it on several new examples. If you find an error, clearly explain the flaw and provide a corrected line of reasoning and a final corrected answer. If the solution is completely sound, state that and re-state the final answer."
        for sol in solutions:
            prompt = f"Original Problem:\\n{problem}\\n\\nProposed Solution to Review:\\n{sol}\\n\\nYour Critical Review and Corrected Solution:"
            try:
                审查、成本 = self.query_llm(
                    prompt=prompt,
                    system=reviewer_system_prompt,
                    temperature=self.review_temperature,
                )
                critiques.append(review)
                total_cost += cost
```

```

except Exception:
    critiques.append("Reviewer failed to provide a critique.")

# === STAGE 3: Synthesize Final Answer ===
synthesis_prompt_parts = [
    f"You are the Editor-in-Chief of a prestigious mathematics journal, responsible for publishing the final, canonical solution to this AIME problem. You have received {self.num_experts} independent attempts and their corresponding critical reviews. Your task is to produce the definitive solution.\n\nProblem:\n{problem}"
]
for i, (sol, crit) in enumerate(zip(solutions, critiques)):
    synthesis_prompt_parts.append(
        f"\n--- 尝试 {i + 1} ---\nSolution: {sol} \n Critique: {crit} \n---"
    )

synthesis_prompt_parts.append(f"\nSYNTHESIS 以及最终判断:\n1. 首先, 简要陈述每个被评审尝试所提出的最终数值答案.\n2. 基于这些批评, 确定哪种方法最可靠, 或者是否所有方法都有缺陷. 解释你的推理.\n3. 构建最终、清晰、逐步、正确的解决方案. 利用尝试中有效部分的洞见, 并纠正任何已识别的错误. {self.output_format_instructions}")

synthesizer_prompt = "\n".join(synthesis_prompt_parts)
synthesizer_system_prompt = "You are a master mathematician and editor, synthesizing multiple reviewed solutions into one canonical, correct answer."

final_response = ""
try:
    final_response, 成本 = self.query_llm(
        prompt=synthesizer_prompt,
        system=synthesizer_system_prompt,
        temperature=self.synthesis_temperature,
    )
    total_cost += cost
except Exception:
    pass # Fallback logic will handle this.

# === Fallback Logic ===
if self._extract_answer(final_response) is None:
    # First, trust the reviewed answers
    reviewed_answers = [self._extract_answer(c) for c in critiques]
    valid_reviewed_answers = [
        答案遍历于 reviewed_answers 若答案非空
    ]

    if valid_reviewed_answers:
        most_common_answer = 计数器(valid_reviewed_answers).most_common(1)[0][0]
        final_response += f"\n\n[Fallback to Majority Vote on Reviewed Solutions]\n\n\boxed{{{most_common_answer}}}"
    否则:
        # If reviews didn't produce answers, check original solutions
        original_answers = [self._extract_answer(s) for s in solutions]
        valid_original_answers = [
            ans for ans in original_answers if ans is not None
        ]
        if valid_original_answers:
            most_common_answer = 计数器(valid_original_answers).most_common(1)[0][0]
            final_response += f"\n\n[Fallback to Majority Vote on Original Solutions]\n\n\boxed{{{most_common_answer}}}"
        否则:
            # Ultimate fallback
            final_response += "\n\n[Fallback] Could not determine a final answer from any stage.\n\n\boxed{000}"

    返回 final_response, total_cost

# 进化-块-结束

定义 run_experiment (** 关键字参数): 从工具模块导入 query_llm、
create_call_limited_query_llm; 从函数工具模块导入偏函数

# Create base query_llm function
base_query_llm = partial(query_llm, model_name=kwargs["model_name"])

# Wrap it with call limiting (max 10 calls per forward pass)
limited_query_llm = create_call_limited_query_llm(
    base_query_llm,
    max_calls=kwargs["max_calls"],
)

准确率, cost_total, 已处理, num_llm_calls, 数据框 = agent_evaluation (
    Agent, limited_query_llm, year=kwargs["year"]
)
return accuracy, cost_total, processed, num_llm_calls, df

```

清单 3 | ShinkaEvolve 发现的 AIME 智能体脚手架设计。

C.3. ALE-Bench 问题

C.3.1. ALE-Bench LITE 任务: ahc039

```
// EVOLVE-BLOCK-START
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>
#include <random>
#include <set>
#include <unordered_set>
#include <cmath>
#include <iomanip>
#include <numeric> // For std::iota
#include <string>
#include <map>

// === MACROS AND CONSTANTS ===
const int MAX_COORD_VAL = 100000;
const int MAX_VERTICES = 1000;
const int MAX_PERIMETER = 400000;
const double TIME_LIMIT_SECONDS_SAFETY_MARGIN = 0.1; // Increased safety margin
double ACTUAL_TIME_LIMIT_SECONDS = 2.0;

// === RANDOM NUMBER GENERATION ===
struct XorShift {
    uint64_t x;
    XorShift() : x(std::chrono::steady_clock::now().time_since_epoch().count() ^ ((uint64_t)std::random_device()
    () << 32) ^ std::random_device()()) {}
    uint64_t next() {
        x ^= x << 13;
        x ^= x >> 7;
        x ^= x << 17;
        return x;
    }
    int next_int(int n) { if (n <= 0) return 0; return next() % n; }
    int next_int(int a, int b) { if (a > b) return a; return a + next_int(b - a + 1); }
    double next_double() { return next() / (double)UINT64_MAX; }
};
XorShift rng;

// === TIMER ===
struct Timer {
    std::chrono::steady_clock::time_point start_time;
    Timer() { reset(); }
    void reset() { start_time = std::chrono::steady_clock::now(); }
    double elapsed() const {
        auto now = std::chrono::steady_clock::now();
        return std::chrono::duration_cast<std::chrono::duration<double>>(now - start_time).count();
    }
};
Timer global_timer;

// === GEOMETRIC STRUCTURES ===
struct Point {
    int x, y;
    bool operator<(const Point& other) const {
        if (x != other.x) return x < other.x;
        return y < other.y;
    }
    bool operator==(const Point& other) const {
        return x == other.x && y == other.y;
    }
    Point operator-(const Point& other) const {
        return {x - other.x, y - other.y};
    }
};

struct PointHash {
    std::size_t operator()(const Point& p) const {
        auto h1 = std::hash<int>{}(p.x);
        auto h2 = std::hash<int>{}(p.y);
        // Combining hashes: simple XOR might not be best, but often good enough.
        // For Point, a common way is boost::hash_combine.
        // h1 ^ (h2 << 1) is a common way that's okay.
        return h1 ^ (h2 << 1);
    }
};

long long cross_product(Point a, Point b) {
    return (long long)a.x * b.y - (long long)a.y * b.x;
}

// 结构体 Fish { 点 p: 整数类型; // 1 表示鲭鱼, -1 表示沙丁鱼 }; 标准库
// 向量 <Fish> all_fish_structs;

// === KD-TREE ===
struct KDNode {
    Point pt;
    int axis;
};
```

```

KDNode *left = nullptr, *right = nullptr;
int fish_struct_idx = -1;
// Subtree bounding box
int min_x, max_x, min_y, max_y;
// Subtree counts
int m_cnt = 0, s_cnt = 0;
};
KDNode* fish_kdtree_root = nullptr;

KDNode* build_kdtree(std::vector<int>& point_indices, int l, int r, int axis) {
    if (l > r) return nullptr;
    int mid = l + (r - l) / 2;

    标准库 nth_element (point_indices.开始 () +1, point_indices.开始 () + 中间, point_indices.开始 () +r+1,
        [&](int a_idx, int b_idx) {
            const Point& pa = all_fish_structs[a_idx].p;
            const Point& pb = all_fish_structs[b_idx].p;
            if (axis == 0) return pa.x < pb.x;
            return pa.y < pb.y;
        });

    KDNode* node = new KDNode();
    node->fish_struct_idx = point_indices[mid];
    node->pt = all_fish_structs[node->fish_struct_idx].p;
    node->axis = axis;

    // Recurse
    node->left = build_kdtree(point_indices, l, mid - 1, 1 - axis);
    node->right = build_kdtree(point_indices, mid + 1, r, 1 - axis);

    // Initialize subtree bbox to this point
    node->min_x = node->max_x = node->pt.x;
    node->min_y = node->max_y = node->pt.y;
    // Initialize counts with this node's fish
    if (all_fish_structs[node->fish_struct_idx].type == 1) node->m_cnt = 1;
    else node->s_cnt = 1;

    // 合并子节点 如果 (节
    点->左) {
        node->min_x = std::min(node->min_x, node->left->min_x);
        node->max_x = std::max(node->max_x, node->left->max_x);
        node->min_y = std::min(node->min_y, node->left->min_y);
        node->max_y = std::max(node->max_y, node->left->max_y);
        node->m_cnt += node->left->m_cnt;
        node->s_cnt += node->left->s_cnt;
    }
    if (node->right) {
        node->min_x = std::min(node->min_x, node->right->min_x);
        node->max_x = std::max(node->max_x, node->right->max_x);
        node->min_y = std::min(node->min_y, node->right->min_y);
        node->max_y = std::max(node->max_y, node->right->max_y);
        node->m_cnt += node->right->m_cnt;
        node->s_cnt += node->right->s_cnt;
    }
    return node;
}

void delete_kdtree(KDNode* node) { // Recursively delete KD-tree nodes
    if (!node) return;
    delete_kdtree(node->left);
    delete_kdtree(node->right);
    delete node;
}

// === POLYGON UTILITIES ===
long long calculate_perimeter(const std::vector<Point>& poly) {
    if (poly.size() < 2) return 0;
    long long perimeter = 0;
    for (size_t i = 0; i < poly.size(); ++i) {
        const Point& p1 = poly[i];
        const Point& p2 = poly[(i + 1) % poly.size()];
        perimeter += std::abs(p1.x - p2.x) + std::abs(p1.y - p2.y);
    }
    return perimeter;
}

bool is_on_segment(Point p, Point seg_a, Point seg_b) {
    if (cross_product(seg_b - seg_a, p - seg_a) != 0) return false; // Not collinear
    return std::min(seg_a.x, seg_b.x) <= p.x && p.x <= std::max(seg_a.x, seg_b.x) &&
        std::min(seg_a.y, seg_b.y) <= p.y && p.y <= std::max(seg_a.y, seg_b.y);
}

bool is_inside_polygon_wn(Point p, const std::vector<Point>& polygon) {
    int n = polygon.size();
    if (n < 3) return false;

    // 首先检查是否在边界boundary first
    for (int i = 0; i < n; ++i) {
        if (is_on_segment(p, polygon[i], polygon[(i + 1) % n])) return true;
    }

    int wn = 0; // Winding number
    for (int i = 0; i < n; ++i) {

```

```

    Point p1 = polygon[i];
    Point p2 = polygon[(i + 1) % n];
    if (p1.y <= p.y) { // Start y <= P.y
        if (p2.y > p.y && cross_product(p2 - p1, p - p1) > 0) { // An upward crossing, P is left of edge
            wn++;
        }
    } else { // Start y > P.y
        if (p2.y <= p.y && cross_product(p2 - p1, p - p1) < 0) { // A downward crossing, P is right of edge
            wn--;
        }
    }
}
return wn != 0; // wn != 0 means inside; wn == 0 means outside.
}

// Calculate score from scratch by checking all fish
long long point_segment_dist_sq_ortho(Point p, Point a, Point b) {
    long long dx, dy;
    if (a.x == b.x) { // Vertical segment
        dx = p.x - a.x;
        if (p.y < std::min(a.y, b.y)) {
            dy = p.y - std::min(a.y, b.y);
        } else if (p.y > std::max(a.y, b.y)) {
            dy = p.y - std::max(a.y, b.y);
        } else {
            dy = 0;
        }
    } else { // Horizontal segment
        dy = p.y - a.y;
        if (p.x < std::min(a.x, b.x)) {
            dx = p.x - std::min(a.x, b.x);
        } else if (p.x > std::max(a.x, b.x)) {
            dx = p.x - std::max(a.x, b.x);
        } else {
            dx = 0;
        }
    }
    return dx * dx + dy * dy;
}

void calculate_score_from_scratch(const std::vector<Point>& poly, int& m_count, int& s_count) {
    m_count = 0; s_count = 0;
    if (poly.size() < 3) return; // Not a valid polygon for containment
    for (const auto& fish_s : all_fish_structs) {
        if (is_inside_polygon_wn(fish_s.p, poly)) {
            if (fish_s.type == 1) m_count++;
            else s_count++;
        }
    }
}

// Calculate fish counts in a given rectangle using KD-tr
void calculate_score_delta_for_rectangle (KDNode* 节点, 整数 r_min_x, 整数 r_max_x, 整数 r_min_y, 整数 r_max_y,
int& delta_m, int& delta_s) {
    delta_m = 0; delta_s = 0;

    if (!node || r_min_x > r_max_x || r_min_y > r_max_y) { // Invalid rectangle
        return;
    }

    // Iterative KD-tree traversal with subtree bbox pruning and whole-subtree aggregation.
    std::vector<KDNode*> stack;
    stack.reserve(64); // Reasonable reserve size for typical KD-tree depth
    stack.push_back(node);

    while (!stack.empty()) {
        KDNode* current_node = stack.back();
        stack.pop_back();
        if (!current_node) continue;

        // Disjoint?
        if (current_node->max_x < r_min_x || current_node->min_x > r_max_x || current_node->max_y < r_min_y ||
            current_node->min_y > r_max_y) {
            continue;
        }
        // Fully inside?
        if (r_min_x <= current_node->min_x && current_node->max_x <= r_max_x && r_min_y <= current_node->min_y
            && current_node->max_y <= r_max_y) {
            delta_m += current_node->m_cnt;
            delta_s += current_node->s_cnt;
            continue;
        }
        // Partial overlap: account this node's point, then traverse children
        const Point& pt = current_node->pt;
        if (pt.x >= r_min_x && pt.x <= r_max_x && pt.y >= r_min_y && pt.y <= r_max_y) {
            if (all_fish_structs[current_node->fish_struct_idx].type == 1) ++delta_m;
            else ++delta_s;
        }
        if (current_node->left) stack.push_back(current_node->left);
        if (current_node->right) stack.push_back(current_node->right);
    }
}

// Check intersection between two orthogonal segments p1s-p1e and p2s-p2e
bool segments_intersect(Point p1s, Point p1e, Point p2s, Point p2e) {

```

```

// Normalize segments (sort endpoints to simplify overlap checks)
if (p1s.x == p1e.x) { if (p1s.y > p1e.y) std::swap(p1s.y, p1e.y); } // Vertical, sort by y
else { if (p1s.x > p1e.x) std::swap(p1s.x, p1e.x); } // Horizontal, sort by x
if (p2s.x == p2e.x) { if (p2s.y > p2e.y) std::swap(p2s.y, p2e.y); }
else { if (p2s.x > p2e.x) std::swap(p2s.x, p2e.x); }

bool seg1_is_H = (p1s.y == p1e.y);
bool seg2_is_H = (p2s.y == p2e.y);

if (seg1_is_H == seg2_is_H) { // Both horizontal or both vertical
    if (seg1_is_H) { // Both horizontal
        // Check for y-alignment and x-overlap
        return p1s.y == p2s.y && std::max(p1s.x, p2s.x) <= std::min(p1e.x, p2e.x);
    } else { // Both vertical
        // Check for x-alignment and y-overlap
        return p1s.x == p2s.x && std::max(p1s.y, p2s.y) <= std::min(p1e.y, p2e.y);
    }
} else { // One horizontal, one vertical (potential T-junction or cross)
    Point h_s = seg1_is_H ? p1s : p2s; Point h_e = seg1_is_H ? p1e : p2e;
    Point v_s = seg1_is_H ? p2s : p1s; Point v_e = seg1_is_H ? p2e : p1e;
    // Check if intersection point (v_s.x, h_s.y) lies on both segments
    return v_s.x >= h_s.x && v_s.x <= h_e.x && // x_intersect within horizontal segment's x-range
           h_s.y >= v_s.y && h_s.y <= v_e.y; // y_intersect within vertical segment's y-range
}
}

bool check_self_intersection_full(const std::vector<Point>& poly) {
    int M = poly.size();
    if (M < 4) return false;
    for (int i = 0; i < M; ++i) {
        Point p1s = poly[i];
        Point p1e = poly[(i + 1) % M];
        for (int j = i + 2; j < M; ++j) {
            // Skip checking adjacent edges.
            // Edge i is (poly[i], poly[(i+1)%M]). Edge j is (poly[j], poly[(j+1)%M]).
            // If i=0 and j=M-1, then edge i is (poly[0], poly[1]) and edge j is (poly[M-1], poly[0]). These are
            // adjacent.
            if (i == 0 && j == M - 1) continue;

            Point p2s = poly[j];
            Point p2e = poly[(j + 1) % M];
            if (segments_intersect(p1s, p1e, p2s, p2e)) return true;
        }
    }
    return false;
}

// Local self-intersection check: checks edges starting at critical_edge_start_indices_const against all others
bool has_self_intersection_locally(const std::vector<Point>& poly, const std::vector<int>&
    critical_edge_start_indices_const) {
    int M = poly.size();
    if (M < 4) return false;

    std::vector<int> critical_indices = critical_edge_start_indices_const; // Make a copy to modify
    if (critical_indices.empty()) {
        return false;
    }

    std::sort(critical_indices.begin(), critical_indices.end());
    critical_indices.erase(std::unique(critical_indices.begin(), critical_indices.end()), critical_indices.end());

    for (int edge1_s_idx_val_orig : critical_indices) {
        int edge1_s_idx_val = (edge1_s_idx_val_orig % M + M) % M; // Ensure positive modulo
        // No need to check edge1_s_idx_val bounds, it will be in [0, M-1]

        Point p1s = poly[edge1_s_idx_val];
        Point p1e = poly[(edge1_s_idx_val + 1) % M];

        for (int edge2_s_idx = 0; edge2_s_idx < M; ++edge2_s_idx) {
            bool is_adj_or_same_to_p1s_p1e = (edge2_s_idx == edge1_s_idx_val || //
                edge2_s_idx == (edge1_s_idx_val + 1) % M || // edge2 starts where
                (edge2_s_idx + 1) % M == edge1_s_idx_val); // edge2 ends where edge1 starts
            if (is_adj_or_same_to_p1s_p1e) continue;

            Point p2s = poly[edge2_s_idx];
            Point p2e = poly[(edge2_s_idx + 1) % M];
            if (segments_intersect(p1s, p1e, p2s, p2e)) {
                return true;
            }
        }
    }
    return false;
}

bool has_distinct_vertices_unordered(const std::vector<Point>& poly) {
    if (poly.empty()) return true;
    std::unordered_set<Point, PointHash> distinct_pts;
    distinct_pts.reserve(poly.size()); // Pre-allocate for efficiency
    for (const auto& p : poly) {
        if (!distinct_pts.insert(p).second) return false; // Insertion failed, duplicate found
    }
}

```

```

    return true;
}

// Check basic structural validity of the polygon, uses cached perimeter
bool is_polygon_structurally_sound(const std::vector<Point>& poly, long long cached_perimeter) {
    int m = poly.size();
    if (m != 0 && (m < 4 || m > MAX_VERTICES)) return false;
    if (m == 0) return true;

    if (cached_perimeter > MAX_PERIMETER) return false;

    对于 (size_t i=0; i<m; ++i) {
        const Point& p1 = poly[i];
        const Point& p2 = poly[(i + 1) % m];
        // Check coordinate bounds for p1
        if (p1.x < 0 || p1.x > MAX_COORD_VAL || p1.y < 0 || p1.y > MAX_COORD_VAL) return false;
        // The endpoint poly[(i+1)%m] will be checked as p1 in its own iteration,
        // but an explicit check here is also fine for robustness, though slightly redundant.
        if (poly[(i+1)%m].x < 0 || poly[(i+1)%m].x > MAX_COORD_VAL || poly[(i+1)%m].y < 0 || poly[(i+1)%m].y >
            MAX_COORD_VAL) return false;

        // Check axis-parallel and non-zero length edges
        if (p1.x != p2.x && p1.y != p2.y) return false; // Not axis-parallel
        if (p1.x == p2.x && p1.y == p2.y) return false; // Zero-length edge (duplicate consecutive vertices)
    }
    return true;
}

// 在粗网格上使用卡丹算法生成初始多边形 std::vector<Point> create_initial_polygon_kadane() {

    const int GRID_SIZE_KADANE = 350; // Tunable parameter
    const int NUM_VALUES_KADANE = MAX_COORD_VAL + 1;
    // Ensure ACTUAL_CELL_DIM_KADANE is at least 1
    const int ACTUAL_CELL_DIM_KADANE = std::max(1, (NUM_VALUES_KADANE + GRID_SIZE_KADANE - 1) / GRID_SIZE_KADANE);

    标准库向量<标准库向量<长整型>> grid_scores(GRID_SIZE_KADANE, 标准库向量<长整型>(GRID_SIZE_KADANE,
    0));
    for (const auto& fish_s : all_fish_structs) {
        int r = fish_s.p.y / ACTUAL_CELL_DIM_KADANE;
        int c = fish_s.p.x / ACTUAL_CELL_DIM_KADANE;
        r = std::min(r, GRID_SIZE_KADANE - 1); r = std::max(r, 0);
        c = std::min(c, GRID_SIZE_KADANE - 1); c = std::max(c, 0);
        grid_scores[r][c] += fish_s.type; // Mackerel +1, Sardine -1
    }

    long long max_so_far = -3e18; // Sufficiently small number
    int best_r1 = 0, best_c1 = 0, best_r2 = -1, best_c2 = -1;

    // 2D Kadane's algorithm
    for (int c1_idx = 0; c1_idx < GRID_SIZE_KADANE; ++c1_idx) {
        std::vector<long long> col_strip_sum(GRID_SIZE_KADANE, 0);
        for (int c2_idx = c1_idx; c2_idx < GRID_SIZE_KADANE; ++c2_idx) {
            for (int r_idx = 0; r_idx < GRID_SIZE_KADANE; ++r_idx) {
                col_strip_sum[r_idx] += grid_scores[r_idx][c2_idx];
            }

            // 1D Kadane's on col_strip_sum
            long long current_strip_val = 0;
            int current_r1_1d = 0;
            for (int r2_idx_1d = 0; r2_idx_1d < GRID_SIZE_KADANE; ++r2_idx_1d) {
                long long val_here = col_strip_sum[r2_idx_1d];
                if (current_strip_val > 0 && current_strip_val + val_here > 0) { // Extend if sum remains
                    positive
                        current_strip_val += val_here;
                } else { // Start new subarray
                    current_strip_val = val_here;
                    current_r1_1d = r2_idx_1d;
                }

                if (current_strip_val > max_so_far) {
                    max_so_far = current_strip_val;
                    best_r1 = current_r1_1d;
                    best_r2 = r2_idx_1d;
                    best_c1 = c1_idx;
                    best_c2 = c2_idx;
                }
            }
        }
    }

    std::vector<Point> default_poly = {{0,0}, {1,0}, {1,1}, {0,1}}; // 最小有效多边形

    // If no positive sum found, or issue, find best single cell
    if (best_r2 == -1 || max_so_far <= 0) {
        max_so_far = -3e18; // Reset search for single best cell
        bool found_cell = false;
        for (int r=0; r<GRID_SIZE_KADANE; ++r) for (int c=0; c<GRID_SIZE_KADANE; ++c) {
            如果 (grid_scores[r][c] > max_so_far) {
                max_so_far = grid_scores[r][c];
                best_r1 = r; best_r2 = r; // Single cell
                best_c1 = c; best_c2 = c;
                found_cell = true;
            }
        }
    }
}

```

```

    if (!found_cell || max_so_far <= 0) return default_poly; // Still no good cell, return default
}

// Convert grid cell indices to actual coordinates
int x_start = best_c1 * ACTUAL_CELL_DIM_KADANE;
int y_start = best_r1 * ACTUAL_CELL_DIM_KADANE;
int x_end = (best_c2 + 1) * ACTUAL_CELL_DIM_KADANE - 1;
int y_end = (best_r2 + 1) * ACTUAL_CELL_DIM_KADANE - 1;

// Clamp coordinates to valid range
x_start = std::max(0, std::min(MAX_COORD_VAL, x_start));
y_start = std::max(0, std::min(MAX_COORD_VAL, y_start));
x_end = std::max(x_start, std::min(MAX_COORD_VAL, x_end)); // Ensure x_end >= x_start
y_end = std::max(y_start, std::min(MAX_COORD_VAL, y_end)); // Ensure y_end >= y_start

// Ensure non-zero dimensions for the polygon, minimum 1x1 actual area
if (x_start == x_end) {
    if (x_start < MAX_COORD_VAL) x_end = x_start + 1;
    else if (x_start > 0) x_start = x_start - 1; // Can't expand right, try expand left
    else return default_poly; // Single point at MAX_COORD_VAL, cannot form 1x1
}
if (y_start == y_end) {
    if (y_start < MAX_COORD_VAL) y_end = y_start + 1;
    else if (y_start > 0) y_start = y_start - 1;
    else return default_poly;
}
// After adjustment, if still degenerate, use default. This is rare.
if (x_start == x_end || y_start == y_end) return default_poly;

std::vector<Point> initial_poly = { {x_start, y_start}, {x_end, y_start}, {x_end, y_end}, {x_start, y_end} }; 返回 initial_poly;

// === SIMULATED ANNEALING ===
struct SASState {
    std::vector<Point> poly;
    int m_count;
    int s_count;
    long long perimeter_cache; // Added cache for perimeter

    SASState(): m_count(0), s_count(0), perimeter_cache(0) {} // 初始化 perimeter_cache

    long long get_objective_score() const {
        return std::max(0LL, (long long)m_count - s_count + 1);
    }
    double get_raw_objective_score() const { // Used for SA acceptance probability
        return (double)m_count - s_count;
    }
};

// Calculates signed area * 2 of a polygon (shoelace formula)
long long polygon_signed_area_times_2(const std::vector<Point>& poly) {
    if (poly.size() < 3) return 0;
    long long area_sum = 0;
    for (size_t i = 0; i < poly.size(); ++i) {
        const Point& p1 = poly[i];
        const Point& p2 = poly[(i + 1) % poly.size()];
        area_sum += (long long)(p1.x - p2.x) * (p1.y + p2.y); // (x1-x2)(y1+y2) variant
    }
    return area_sum; // Positive for CCW, negative for CW
}

std::vector<int> sa_critical_edge_indices_cache; // Cache for local intersection check

// Guide coordinates for SA moves
std::vector<int> static_x_guides;
std::vector<int> static_y_guides;
std::vector<int> best_poly_x_guides;
std::vector<int> best_poly_y_guides;

void update_best_poly_guides(const SASState& new_best_state) {
    best_poly_x_guides.clear();
    best_poly_y_guides.clear();
    if (new_best_state.poly.empty()) return;

    std::set<int> temp_x_set, temp_y_set;
    for (const auto& p : new_best_state.poly) {
        temp_x_set.insert(p.x);
        temp_y_set.insert(p.y);
    }
    best_poly_x_guides.assign(temp_x_set.begin(), temp_x_set.end());
    best_poly_y_guides.assign(temp_y_set.begin(), temp_y_set.end());
}

void simulated_annealing_main() {
    SASState current_state;
    current_state.poly = create_initial_polygon_kadane();
    calculate_score_from_scratch(current_state.poly, current_state.m_count, current_state.s_count);
    current_state.perimeter_cache = calculate_perimeter(current_state.poly); // Calculate initial perimeter

    std::vector<Point> default_tiny_poly = {{0,0}, {1,0}, {1,1}, {0,1}};

```

```

// Ensure initial polygon is valid, otherwise use default
bool current_poly_initial_valid = is_polygon_structurally_sound(current_state.poly, current_state.
    perimeter_cache) &&
    current_state.poly.size() >= 4 &&
    has_distinct_vertices_unordered(current_state.poly) &&
    !check_self_intersection_full(current_state.poly);

如果 (if) !current_poly_initial_valid {
    current_state.poly = default_tiny_poly;
    calculate_score_from_scratch(current_state.poly, current_state.m_count, current_state.s_count);
    current_state.perimeter_cache = calculate_perimeter(current_state.poly); // Update perimeter for
default
}

SAState best_state = current_state;
update_best_poly_guides(best_state);

// Prepare static guide coordinates from fish locations
std::set<int> sx_set, sy_set;
for(const auto& f_s : all_fish_structs) {
    sx_set.insert(f_s.p.x); sx_set.insert(std::max(0, f_s.p.x-1)); sx_set.insert(std::min(MAX_COORD_VAL, f_s.
        p.x+1));
    sy_set.insert(f_s.p.y); sy_set.insert(std::max(0, f_s.p.y-1)); sy_set.insert(std::min(MAX_COORD_VAL, f_s.
        p.y+1));
}
sx_set.insert(0); sx_set.insert(MAX_COORD_VAL); // Boundary guides
sy_set.insert(0); sy_set.insert(MAX_COORD_VAL);

static_x_guides.assign(sx_set.begin(), sx_set.end());
static_y_guides.assign(sy_set.begin(), sy_set.end());

double start_temp = 150.0;
double end_temp = 0.01;

long long current_signed_area = polygon_signed_area_times_2(current_state.poly);
if (current_signed_area == 0 && current_state.poly.size() >=3) {
    current_signed_area = 1; // Avoid issues with zero area for sign logic
}

sa_critical_edge_indices_cache.reserve(10); // Max expected critical edges for current moves

当 (while) global_timer.elapsed() < ACTUAL_TIME_LIMIT_SECONDS 时 {
    double time_ratio = global_timer.elapsed() / ACTUAL_TIME_LIMIT_SECONDS;
    double temperature = start_temp * std::pow(end_temp / start_temp, time_ratio);
    // Fine-tune temperature near end or if it drops too fast
    if (temperature < end_temp && time_ratio < 0.95) temperature = end_temp;
    if (time_ratio > 0.95 && temperature > end_temp * 0.1) temperature = end_temp * 0.1; // Lower temp
    aggressively at the very end

    if (current_state.poly.size() < 4) { // Should not happen if logic is correct, but as a safeguard
        current_state.poly = default_tiny_poly;
        calculate_score_from_scratch(current_state.poly, current_state.m_count, current_state.s_count);
        current_state.perimeter_cache = calculate_perimeter(current_state.poly); // Update perimeter
        current_signed_area = polygon_signed_area_times_2(current_state.poly);
        if (current_signed_area == 0 && current_state.poly.size() >=3) current_signed_area = 1;
    }

    SAState candidate_state = current_state; // Copy current state
    sa_critical_edge_indices_cache.clear();

    整型 (int) move_type_roll = rng.next_int(100);

    // Base probabilities for moves
    int targeted_move_prob = 35;
    int move_edge_prob = 35;
    int add_bulge_prob = 10;
    // simplify gets 20%

    bool near_vertex_limit = candidate_state.poly.size() + 2 > MAX_VERTICES;
    bool near_perimeter_limit = false;
    // Check perimeter using candidate_state's cached value
    if (candidate_state.poly.size() > 200 && candidate_state.perimeter_cache > MAX_PERIMETER * 0.9) {
        near_perimeter_limit = true;
    }

    // Adjust move probabilities based on polygon size/perimeter
    if (near_vertex_limit || near_perimeter_limit) {
        add_bulge_prob = 0;
        targeted_move_prob = 40;
        move_edge_prob = 40; // simplify is 20
    } else if (candidate_state.poly.size() > 400) {
        add_bulge_prob = 5;
        targeted_move_prob = 35;
        move_edge_prob = 35; // simplify is 25
    }

    int p_targeted = targeted_move_prob;
    int p_move_edge = p_targeted + move_edge_prob;
    int p_add_bulge = p_move_edge + add_bulge_prob;

    布尔型 (bool) move_made = 假
    (false);

    // 吸附到参考坐标的概率

```



```

double prob_dynamic_guide_snap = 0.20 + 0.20 * time_ratio;
double prob_static_guide_snap_if_not_dynamic = 0.75;

if (move_type_roll < p_targeted && candidate_state.poly.size() >= 4) { // Targeted Edge Move
    bool target_mackerel = rng.next_double() < 0.7;
    int n_fish_half = all_fish_structs.size() / 2;
    int fish_idx = target_mackerel ? rng.next_int(n_fish_half) : n_fish_half + rng.next_int(n_fish_half)
;

    const auto& target_fish = all_fish_structs[fish_idx];
    bool is_inside = is_inside_polygon_wn(target_fish.p, candidate_state.poly);

    if ((target_fish.type == 1) == is_inside) {
        move_made = false; goto end_move_attempt_label;
    }

    long long min_dist_sq = -1;
    int best_edge_idx = -1;
    for (size_t i = 0; i < candidate_state.poly.size(); ++i) {
        long long d_sq = point_segment_dist_sq_ortho(target_fish.p, candidate_state.poly[i],
candidate_state.poly[(i+1)%candidate_state.poly.size()]);
        if (best_edge_idx == -1 || d_sq < min_dist_sq) {
            min_dist_sq = d_sq;
            best_edge_idx = i;
        }
    }
    if (best_edge_idx == -1) { move_made = false; goto end_move_attempt_label; }

    int edge_idx = best_edge_idx;
    Point p1_orig = candidate_state.poly[edge_idx];
    Point p2_orig = candidate_state.poly[(edge_idx + 1) % candidate_state.poly.size()];

    int new_coord_val;
    if (p1_orig.x == p2_orig.x) { new_coord_val = target_fish.p.x; }
    else { new_coord_val = target_fish.p.y; }

    new_coord_val = 标准库 (std) :: 最大值 (max) (0, 标准库 (std) :: min(MAX_COORD_VAL, new_coord_val) );

    int cur_delta_m=0, cur_delta_s=0;
    if (p1_orig.x == p2_orig.x) { // Vertical edge
        if (new_coord_val == p1_orig.x) {move_made = false; goto end_move_attempt_label;}

        int query_min_x, query_max_x;
        if (new_coord_val > p1_orig.x) { query_min_x = p1_orig.x + 1; query_max_x = new_coord_val; }
        else { query_min_x = new_coord_val; query_max_x = p1_orig.x - 1; }

        calculate_score_delta_for_rectangle(
            fish_kdtree_root, query_min_x, query_max_x,
            std::min(p1_orig.y, p2_orig.y), std::max(p1_orig.y, p2_orig.y),
            cur_delta_m, cur_delta_s);

        int sign = (new_coord_val > p1_orig.x) ? 1 : -1;
        if (p1_orig.y > p2_orig.y) sign *= -1;
        if (current_signed_area < 0) sign *= -1;

        candidate_state.poly[edge_idx].x = new_coord_val;
        candidate_state.poly[(edge_idx + 1) % candidate_state.poly.size()].x = new_coord_val;
        candidate_state.m_count += sign * cur_delta_m;
        candidate_state.s_count += sign * cur_delta_s;
    } else { // Horizontal edge
        if (new_coord_val == p1_orig.y) {move_made = false; goto end_move_attempt_label;}

        int query_min_y, query_max_y;
        if (new_coord_val > p1_orig.y) { query_min_y = p1_orig.y + 1; query_max_y = new_coord_val; }
        else { query_min_y = new_coord_val; query_max_y = p1_orig.y - 1; }

        calculate_score_delta_for_rectangle(
            fish_kdtree_root, std::min(p1_orig.x, p2_orig.x), std::max(p1_orig.x, p2_orig.x),
            query_min_y, query_max_y,
            cur_delta_m, cur_delta_s);

        int sign = (new_coord_val < p1_orig.y) ? 1 : -1;
        if (p1_orig.x > p2_orig.x) sign *= -1;
        if (current_signed_area < 0) sign *= -1;

        candidate_state.poly[edge_idx].y = new_coord_val;
        candidate_state.poly[(edge_idx + 1) % candidate_state.poly.size()].y = new_coord_val;
        candidate_state.m_count += sign * cur_delta_m;
        candidate_state.s_count += sign * cur_delta_s;
    }
    int M_cand = candidate_state.poly.size();
    sa_critical_edge_indices_cache.push_back((edge_idx - 1 + M_cand) % M_cand);
    sa_critical_edge_indices_cache.push_back(edge_idx);
    sa_critical_edge_indices_cache.push_back((edge_idx + 1) % M_cand);
    move_made = true;
} else if (move_type_roll < p_move_edge && candidate_state.poly.size() >= 4) { // Move Edge
    int edge_idx = rng.next_int(candidate_state.poly.size());
    Point p1_orig = candidate_state.poly[edge_idx];
    Point p2_orig = candidate_state.poly[(edge_idx + 1) % candidate_state.poly.size()];

    int new_coord_val = -1;
    int cur_delta_m=0, cur_delta_s=0;
    bool coord_selected_successfully = false;

    // Determine which guides are relevant (X or Y)

```

```

const std::vector<int>* relevant_dyn_guides = (p1_orig.x == p2_orig.x) ? &best_poly_x_guides : &
best_poly_y_guides;
const std::vector<int>* relevant_static_guides = (p1_orig.x == p2_orig.x) ? &static_x_guides : &
static_y_guides;

// Try snapping to dynamic (best poly) guides
if (!relevant_dyn_guides->empty() && rng.next_double() < probab_dynamic_guide_snap) {
    new_coord_val = (*relevant_dyn_guides)[rng.next_int(relevant_dyn_guides->size())];
    coord_selected_successfully = true;
}
// If not, try snapping to static (fish) guides
if (!coord_selected_successfully) {
    if (!relevant_static_guides->empty() && rng.next_double() <
    probab_static_guide_snap_if_not_dynamic) {
        new_coord_val = (*relevant_static_guides)[rng.next_int(relevant_static_guides->size())];
        coord_selected_successfully = true;
    }
}
// If still not selected, use random displacement
if (!coord_selected_successfully) {
    double step_factor = std::max(0.1, 1.0 - time_ratio * 0.95); // Step size decreases over time
    int base_step_max = std::max(1, (int)((MAX_COORD_VAL/150.0) * step_factor + 1));
    int random_displacement = rng.next_int(-base_step_max, base_step_max);
    if (time_ratio > 0.75 && rng.next_double() < 0.7) { // Very small steps near end
        random_displacement = rng.next_int(-2,2);
    }
    if (random_displacement == 0) random_displacement = (rng.next_double() < 0.5) ? -1:1;

    if (p1_orig.x == p2_orig.x) new_coord_val = p1_orig.x + random_displacement; // Vertical edge,
move X
否则 (else) new_coord_val = p1_orig.y + random_displacement; // 水平边, 移动 Y 坐标 } new_coord_val = 标准库 (std) :: 最大值
(max) (0, 标准库 (std) :: min(MAX_COORD_VAL, new_coord_val) ); // 限制在边界内

if (p1_orig.x == p2_orig.x) { // Vertical edge: (X_orig, Y_s) to (X_orig, Y_e)
    if (new_coord_val == p1_orig.x) {move_made = false; goto end_move_attempt_label;} // No change

    int query_min_x, query_max_x;
    if (new_coord_val > p1_orig.x) { // Moved right
        query_min_x = p1_orig.x + 1;
        query_max_x = new_coord_val;
    } else { // Moved left (new_coord_val < p1_orig.x)
        query_min_x = new_coord_val;
        query_max_x = p1_orig.x - 1;
    }

    calculate_score_delta_for_rectangle(
        fish_kdtree_root, query_min_x, query_max_x,
        std::min(p1_orig.y, p2_orig.y), std::max(p1_orig.y, p2_orig.y),
        cur_delta_m, cur_delta_s);

    int sign = (new_coord_val > p1_orig.x) ? 1 : -1; // Moving right is positive X change
    if (p1_orig.y > p2_orig.y) sign *= -1; // Correct for edge Y-direction (p1_orig.y to p2_orig.y)
    if (current_signed_area < 0) sign *= -1; // Correct for CW polygon (area < 0)

    candidate_state.poly[edge_idx].x = new_coord_val;
    candidate_state.poly[(edge_idx + 1) % candidate_state.poly.size()].x = new_coord_val;
    candidate_state.m_count += sign * cur_delta_m;
    candidate_state.s_count += sign * cur_delta_s;
} else { // Horizontal edge: (X_s, Y_orig) to (X_e, Y_orig)
    if (new_coord_val == p1_orig.y) {move_made = false; goto end_move_attempt_label;} // No change

    int query_min_y, query_max_y;
    if (new_coord_val > p1_orig.y) { // Moved up (Y increases)
        query_min_y = p1_orig.y + 1;
        query_max_y = new_coord_val;
    } else { // Moved down (Y decreases, new_coord_val < p1_orig.y)
        query_min_y = new_coord_val;
        query_max_y = p1_orig.y - 1;
    }

    calculate_score_delta_for_rectangle(
        fish_kdtree_root, std::min(p1_orig.x, p2_orig.x), std::max(p1_orig.x, p2_orig.x),
        query_min_y, query_max_y,
        cur_delta_m, cur_delta_s);

    int sign = (new_coord_val < p1_orig.y) ? 1 : -1; // Moving "down" (Y decreases) means positive
sign if it
expands area
    if (p1_orig.x > p2_orig.x) sign *= -1; // Correct for edge X-direction (p1_orig.x to p2_orig.x)
    if (current_signed_area < 0) sign *= -1; // Correct for CW polygon

    candidate_state.poly[edge_idx].y = new_coord_val;
    candidate_state.poly[(edge_idx + 1) % candidate_state.poly.size()].y = new_coord_val;
    candidate_state.m_count += sign * cur_delta_m;
    candidate_state.s_count += sign * cur_delta_s;
}
int M_cand = candidate_state.poly.size();
sa_critical_edge_indices_cache.push_back((edge_idx - 1 + M_cand) % M_cand);
sa_critical_edge_indices_cache.push_back(edge_idx);
sa_critical_edge_indices_cache.push_back((edge_idx + 1) % M_cand);
move_made = true;

} else if (move_type_roll < p_add_bulge && candidate_state.poly.size() + 2 <= MAX_VERTICES &&
candidate_state.poly.size() >=4) { // Add Bulge

```

```

int edge_idx = rng.next_int(candidate_state.poly.size());
Point p_s = candidate_state.poly[edge_idx]; // Start point of edge
Point p_e = candidate_state.poly[(edge_idx + 1) % candidate_state.poly.size()]; // End point of edge

int new_coord_val = -1;
bool coord_selected_successfully = false;

const std::vector<int>* relevant_dyn_guides = (p_s.x == p_e.x) ? &best_poly_x_guides : &best_poly_y_guides;
const std::vector<int>* relevant_static_guides = (p_s.x == p_e.x) ? &static_x_guides : &static_y_guides;

// 尝试吸附凸起坐标 如果 (if) !relevant_dyn_guides->empty() 且 (&&) 随机数生成器 (rng) .next_double() <
prob_dynamic_guide_snap {
    new_coord_val = (*relevant_dyn_guides)[rng.next_int(relevant_dyn_guides->size())];
    coord_selected_successfully = true;
}
if (!coord_selected_successfully) {
    if (!relevant_static_guides->empty() && rng.next_double() <
prob_static_guide_snap_if_not_dynamic) {
        new_coord_val = (*relevant_static_guides)[rng.next_int(relevant_static_guides->size())];
        coord_selected_successfully = true;
    }
}
// If not snapped, random depth for bulge
if (!coord_selected_successfully) {
    double depth_factor = std::max(0.1, 1.0 - time_ratio * 0.9);
    int base_depth_max = std::max(1, (int)((MAX_COORD_VAL/300.0) * depth_factor + 1));
    int random_abs_depth = rng.next_int(1, base_depth_max);
    if (time_ratio > 0.75 && rng.next_double() < 0.7) {
        random_abs_depth = rng.next_int(1,2);
    }
    int bulge_dir_sign = (rng.next_double() < 0.5) ? 1 : -1; // Randomly outwards or inwards
relative to edge line
    if (p_s.x == p_e.x) new_coord_val = p_s.x + bulge_dir_sign * random_abs_depth; // Vertical edge,
bulge in X
    否则 new_coord_val = p_s.y + bulge_dir_sign * random_abs_depth; // 水平边, Y 方向凸起 } new_coord_val = std::max(0, std::min(MAX_COORD_VAL,
new_coord_val));

Point v1_mod, v2_mod; // New vertices for the bulge
int cur_delta_m=0, cur_delta_s=0;

if (p_s.x == p_e.x) { // Original edge is vertical
    if (new_coord_val == p_s.x) {move_made = false; goto end_move_attempt_label;} // Bulge is flat
    v1_mod = {new_coord_val, p_s.y}; v2_mod = {new_coord_val, p_e.y};
    // Rectangle for delta score is between X=p_s.x and X=new_coord_val, over Y-span of original
edge
    calculate_score_delta_for_rectangle(
        fish_kdtree_root, std::min(p_s.x, new_coord_val), std::max(p_s.x, new_coord_val),
        std::min(p_s.y, p_e.y), std::max(p_s.y, p_e.y),
        cur_delta_m, cur_delta_s);
    int sign = (new_coord_val > p_s.x) ? 1 : -1; // Bulge to the right of edge is positive X change
    if (p_s.y > p_e.y) sign *= -1; // Correct for edge Y-direction
    if (current_signed_area < 0) sign *= -1; // Correct for CW polygon
    candidate_state.m_count += sign * cur_delta_m;
    candidate_state.s_count += sign * cur_delta_s;
} else { // Original edge is horizontal
    if (new_coord_val == p_s.y) {move_made = false; goto end_move_attempt_label;} // Bulge is flat
    v1_mod = {p_s.x, new_coord_val}; v2_mod = {p_e.x, new_coord_val};
    // Rectangle for delta score is between Y=p_s.y and Y=new_coord_val, over X-span of original
edge
    calculate_score_delta_for_rectangle(
        fish_kdtree_root, std::min(p_s.x, p_e.x), std::max(p_s.x, p_e.x),
        std::min(p_s.y, new_coord_val), std::max(p_s.y, new_coord_val),
        cur_delta_m, cur_delta_s);
    int sign = (new_coord_val < p_s.y) ? 1 : -1; // Bulge "downwards" (Y decreases) means positive
sign if it expands area
    if (p_s.x > p_e.x) sign *= -1; // Correct for edge X-direction
    if (current_signed_area < 0) sign *= -1; // Correct for CW polygon
    candidate_state.m_count += sign * cur_delta_m;
    candidate_state.s_count += sign * cur_delta_s;
}

// Insert new vertices into polygon
auto insert_pos_iter = candidate_state.poly.begin() + (edge_idx + 1);
insert_pos_iter = candidate_state.poly.insert(insert_pos_iter, v1_mod);
candidate_state.poly.insert(insert_pos_iter + 1, v2_mod);

// Mark affected edges/vertices as critical for local intersection check
sa_critical_edge_indices_cache.push_back(edge_idx);
sa_critical_edge_indices_cache.push_back(edge_idx + 1);
sa_critical_edge_indices_cache.push_back(edge_idx + 2);
move_made = true;

} else if (candidate_state.poly.size() > 4) { // Simplify Polygon (remove collinear vertex)
    int R_start_idx = rng.next_int(candidate_state.poly.size()); // Random start for search
    bool simplified_this_turn = false;
    for(int k_offset=0; k_offset < candidate_state.poly.size() ; ++k_offset) {
        int current_poly_size_before_erase = candidate_state.poly.size();
        if (current_poly_size_before_erase <= 4) break; // Cannot simplify further

        int p1_idx = (R_start_idx + k_offset) % current_poly_size_before_erase;
        int p0_idx_old = (p1_idx - 1 + current_poly_size_before_erase) % current_poly_size_before_erase;

```

```

int p2_idx_old = (p1_idx + 1) % current_poly_size_before_erase;

const Point& p0 = candidate_state.poly[p0_idx_old];
const Point& p1 = candidate_state.poly[p1_idx];
const Point& p2 = candidate_state.poly[p2_idx_old];

bool collinear_x = (p0.x == p1.x && p1.x == p2.x);
bool collinear_y = (p0.y == p1.y && p1.y == p2.y);

if (collinear_x || collinear_y) {
    candidate_state.poly.erase(candidate_state.poly.begin() + p1_idx);
    simplified_this_turn = true;

    int M_cand = candidate_state.poly.size();
    int critical_vertex_idx_in_new_poly;
    // Vertex p0 (at p0_idx_old) forms the new corner. Its index in new poly:
    if (p1_idx == 0) { // If p1 was poly[0], p0 was poly[last]. p0 is now poly[new_last]
        critical_vertex_idx_in_new_poly = M_cand - 1;
    } else { // Otherwise, p0's index p1_idx-1 is preserved.
        critical_vertex_idx_in_new_poly = p1_idx - 1;
    }

    if (!candidate_state.poly.empty()) {
        sa_critical_edge_indices_cache.push_back((critical_vertex_idx_in_new_poly - 1 + M_cand)
% M_cand);
        sa_critical_edge_indices_cache.push_back(critical_vertex_idx_in_new_poly);
        sa_critical_edge_indices_cache.push_back((critical_vertex_idx_in_new_poly + 1) % M_cand)
;
    }
    break; // Simplified one vertex, enough for this turn
}
if (!simplified_this_turn) {move_made = false; goto end_move_attempt_label;} // No simplification
found/possible
move_made = true;
}

// After any move, recalculate perimeter for the candidate_state. This occurs only once per candidate.
candidate_state.perimeter_cache = calculate_perimeter(candidate_state.poly);

end_move_attempt_label: // Label for goto if a move is aborted (e.g. no change)
if (!move_made) continue; // No valid move attempted or made

// 使用缓存的周长验证候选多边形, 如果!is_polygon_structurally_sound(candidate_state.poly, candidate_state.perimeter_cache) ||

    candidate_state.poly.size() < 4 ||
    !has_distinct_vertices_unordered(candidate_state.poly)) {
        continue; // Invalid basic structure or duplicate vertices
    }

if (has_self_intersection_locally(candidate_state.poly, sa_critical_edge_indices_cache)) { continue; // 发现自相交 }

// Accept or reject candidate based on SA criteria
double candidate_raw_obj_score = candidate_state.get_raw_objective_score();
double current_raw_obj_score = current_state.get_raw_objective_score();
double score_diff = candidate_raw_obj_score - current_raw_obj_score;

if (score_diff >= 0 || (temperature > 1e-9 && rng.next_double() < std::exp(score_diff / temperature))) {
    current_state = std::move(candidate_state); // Accept move (perimeter_cache is moved as well)
    current_signed_area = polygon_signed_area_times_2(current_state.poly); // Update signed area
    if (current_signed_area == 0 && !current_state.poly.empty() && current_state.poly.size() >= 3)
current_signed_area = 1; // Handle degenerate

    if (current_state.get_objective_score() > best_state.get_objective_score()) {
        best_state = current_state; // New best solution found (perimeter_cache is copied here)
        update_best_poly_guides(best_state); // Update dynamic guides
    }
}
} // End SA loop

// Final validation of the best found state: Recalculate perimeter explicitly for safety
bool needs_reset_to_default = false;
if (!is_polygon_structurally_sound(best_state.poly, calculate_perimeter(best_state.poly)) ||
best_state.poly.size() < 4 ||
!has_distinct_vertices_unordered(best_state.poly) ||
check_self_intersection_full(best_state.poly)) { // Full intersection check on best
needs_reset_to_default = true;
}

if (needs_reset_to_default) { // If best state is invalid, revert to default
    best_state.poly = default_tiny_poly;
    calculate_score_from_scratch(best_state.poly, best_state.m_count, best_state.s_count);
    best_state.perimeter_cache = calculate_perimeter(best_state.poly); // Update for default
}

// If best score is 0, check if default polygon gives >0. (max(0, val+1))
if (best_state.get_objective_score() == 0) {
    SAState temp_default_state; // Create a temporary default state to calculate its score
    temp_default_state.poly = default_tiny_poly;
    calculate_score_from_scratch(temp_default_state.poly, temp_default_state.m_count, temp_default_state.
s_count);
    temp_default_state.perimeter_cache = calculate_perimeter(temp_default_state.poly); // Update for default

```

```

        if (best_state.get_objective_score() < temp_default_state.get_objective_score()) {
            best_state = temp_default_state;
        }
    }

    // Output the best polygon
    std::cout << best_state.poly.size() << "\n";
    for (const auto& p : best_state.poly) {
        std::cout << p.x << " " << p.y << "\n";
    }
}

int main(int argc, char *argv[]) {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    // 允许通过命令行参数覆盖时间限制, 用于本地测试 if(argc > 1) { try {
        ACTUAL_TIME_LIMIT_SECONDS = std::stod(argv[1]);
    } catch (const std::exception& e) { /* keep default if parse fails */ }
    }
    ACTUAL_TIME_LIMIT_SECONDS -= TIME_LIMIT_SECONDS_SAFETY_MARGIN;
    if (ACTUAL_TIME_LIMIT_SECONDS < 0.2) ACTUAL_TIME_LIMIT_SECONDS = 0.2; // Minimum sensible time limit

    // query_rect_indices_cache_kdtree.reserve(2 * 5000 + 500); // Removed: unused
    sa_critical_edge_indices_cache.reserve(10); // Small, for a few critical edges

    int N_half; // Number of mackerels (and sardines)
    std::cin >> N_half;

    all_fish_structs.resize(2 * N_half);
    std::vector<int> fish_indices_for_kdtree(2 * N_half);
    if (2 * N_half > 0) {
        std::iota(fish_indices_for_kdtree.begin(), fish_indices_for_kdtree.end(), 0);
    }

    // 读取鲭鱼 for (inti = 0; i < N_half; ++i) {
        std::cin >> all_fish_structs[i].p.x >> all_fish_structs[i].p.y;
        all_fish_structs[i].type = 1;
    }
    // Read sardines
    for (int i = 0; i < N_half; ++i) {
        std::cin >> all_fish_structs[N_half + i].p.x >> all_fish_structs[N_half + i].p.y;
        all_fish_structs[N_half + i].type = -1;
    }

    // 如果有鱼, 构建 KD 树 if (all_fish_structs.empty())
    {
        (fish_indices_for_kdtree, 0, (int)all_fish_structs.size() - 1, 0);
    }

    simulated_annealing_main();

    // 清理 KD 树内存 if (fish_kdtree_root) delete_kdtree (fish_kdtree_root);

    return 0;
}
// EVOLVE-BLOCK-END

```

清单 4 | ShinkaEvolve 发现的 ahc039 解决方案。

C.3.2. ALE-Bench LITE 任务: ahc025

```
// EVOLVE-BLOCK-START
#include <iostream>
#include <vector>
#include <string>
#include <numeric>
#include <algorithm>
#include <iomanip>
#include <cmath>
#include <set>
#include <map>
#include <chrono>
#include <random>
#include <unordered_map>

// Timer
std::chrono::steady_clock::time_point program_start_time;
std::chrono::milliseconds time_limit_ms(1850);

// 全局问题参数 int N_items_global, D_groups_global, Q_total_global; int
queries_made = 0;

std::mt19937 rng_engine;

// Query Manager with optimized caching
class QueryManager {
private:
    int N, Q;
    int& queries_made_ref;
    std::vector<char> cmp1_flat; // flat N*N storage for 1v1 comparisons
    std::unordered_map<uint32_t, char> cmp1v2; // for 1v2 comparisons
    std::mt19937& rng;

    inline uint32_t key1v2(int a, int b, int c) const {
        int mn = std::min(b, c), mx = std::max(b, c);
        return (static_cast<uint32_t>(a) << 16) | (static_cast<uint32_t>(mn) << 8) | static_cast<uint32_t>(mx);
    }

    char perform_query_actual(const std::vector<int>& L_items, const std::vector<int>& R_items) {
        queries_made_ref++;
        std::cout << L_items.size() << " " << R_items.size();
        for (int item_idx : L_items) {
            std::cout << " " << item_idx;
        }
        for (int item_idx : R_items) {
            std::cout << " " << item_idx;
        }
        std::cout << std::endl;

        char result_char;
        std::cin >> result_char;
        return result_char;
    }

public:
    QueryManager(int N_, int Q_, int& qm, std::mt19937& r) : N(N_), Q(Q_), queries_made_ref(qm), rng(r) {
        cmp1_flat.assign(N * N, 0);
        cmp1v2.reserve(N * N / 4 + 10);
    }

    char compare1(int a, int b) {
        if (a == b) return '=';
        int mn = std::min(a, b), mx = std::max(a, b);
        char cached = cmp1_flat[mn * N + mx];
        if (cached != 0) {
            if (a == mn) return cached;
            return (cached == '<' ? '>' : (cached == '>' ? '<' : '='));
        }
        if (queries_made_ref >= Q) return '=';

        char res = perform_query_actual({a}, {b});
        if (a == mn) {
            cmp1_flat[mn * N + mx] = res;
        } else {
            if (res == '<') cmp1_flat[mn * N + mx] = '>';
            else if (res == '>') cmp1_flat[mn * N + mx] = '<';
            else cmp1_flat[mn * N + mx] = '=';
        }
        return res;
    }

    char compare1v2(int item_curr, int item_prev, int item_s_aux) {
        if (item_curr == item_prev || item_curr == item_s_aux || item_prev == item_s_aux) {
            if (item_prev == item_s_aux) return compare1(item_curr, item_prev);
            if (item_curr == item_prev) return compare1(item_curr, item_s_aux);
            return compare1(item_curr, item_prev);
        }
        uint32_t key = key1v2(item_curr, item_prev, item_s_aux);
        auto it = cmp1v2.find(key);
        if (it != cmp1v2.end()) return it->second;
        if (queries_made_ref >= Q) return '=';
        char res = perform_query_actual({item_curr}, {item_prev, item_s_aux});
        cmp1v2.emplace(key, res);
        return res;
    }
};
```

```

    }

    void exhaust_queries() {
        if (N >= 2) {
            int a = 0, b = 1;
            while (queries_made_ref < Q) {
                perform_query_actual({a}, {b});
                ++b;
                if (b == a) ++b;
                if (b >= N) {
                    b = 0;
                    a = (a + 1) % N;
                    if (b == a) b = (b + 1) % N;
                }
            }
        }
    }
};

// 权重估计模块 class WeightEstimator {
private:

    static constexpr long long BASE_WEIGHT = 100000;
    static constexpr int FACTOR_GT = 200;
    static constexpr int FACTOR_LT = 50;
    static constexpr int FACTOR_XJ_FALLBACK = 100;

    QueryManager& qm;
    int N, D, Q;

    double estimate_log2(double val) {
        return (val <= 1.0) ? 0.0 : std::log2(val);
    }

    int calculate_query_cost(int N_val, int k_pivots) {
        if (k_pivots <= 0) return 0;
        if (k_pivots == 1) return std::max(0, N_val - 1);
        double cost = 0;
        cost += k_pivots * estimate_log2(k_pivots);
        for (int j = 2; j < k_pivots; ++j) {
            if (j - 1 > 0) cost += estimate_log2(j - 1);
        }
        cost += (N_val - k_pivots) * estimate_log2(k_pivots);
        return static_cast<int>(std::ceil(cost));
    }

    void merge_sort_pivots(std::vector<int>& pivots, int left, int right) {
        if (left >= right) return;
        int mid = (left + right) / 2;
        merge_sort_pivots(pivots, left, mid);
        merge_sort_pivots(pivots, mid + 1, right);

        int n1 = mid - left + 1, n2 = right - mid;
        std::vector<int> L(n1), R(n2);
        for (int i = 0; i < n1; ++i) L[i] = pivots[left + i];
        for (int j = 0; j < n2; ++j) R[j] = pivots[mid + 1 + j];

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            char cmp = qm.compare1(L[i], R[j]);
            if (cmp == '<' || cmp == '=') pivots[k++] = L[i++];
            else pivots[k++] = R[j++];
        }
        while (i < n1) pivots[k++] = L[i++];
        while (j < n2) pivots[k++] = R[j++];
    }

public:
    WeightEstimator(QueryManager& qm_, int N_, int D_, int Q_) : qm(qm_), N(N_), D(D_), Q(Q_) {}

    std::vector<long long> estimate_weights() {
        std::vector<long long> weights(N, BASE_WEIGHT);

        // Determine pivot count
        int k_pivots = (N > 0) ? 1 : 0;
        if (N > 1) {
            for (int k = N; k >= 1; --k) {
                if (calculate_query_cost(N, k) <= Q) {
                    k_pivots = k;
                    break;
                }
            }
        }
        k_pivots = std::min(k_pivots, N);

        if (k_pivots == 0) return weights;

        // Select and sort pivots
        std::vector<int> pivots(k_pivots);
        std::vector<int> indices(N);
        std::iota(indices.begin(), indices.end(), 0);
        std::shuffle(indices.begin(), indices.end(), rng_engine);
        for (int i = 0; i < k_pivots; ++i) pivots[i] = indices[i];

        if (k_pivots >= 2) {

```

```

    merge_sort_pivots(pivots, 0, k_pivots - 1);
}

// Estimate pivot weights
if (k_pivots == 1) {
    weights[pivots[0]] = BASE_WEIGHT;
    for (int i = 0; i < N; ++i) {
        if (i == pivots[0]) continue;
        char res = qm.compare1(i, pivots[0]);
        if (res == '=') weights[i] = BASE_WEIGHT;
        else if (res == '<') weights[i] = std::max(1LL, BASE_WEIGHT * FACTOR_LT / 100);
        else weights[i] = std::max(1LL, BASE_WEIGHT * FACTOR_GT / 100);
    }
} else {
    // 多枢轴估计权重 [pivots[0]] = BASE_WEIGHT;

    // Handle p1
    char res_p1 = qm.compare1(pivots[1], pivots[0]);
    if (res_p1 == '=') weights[pivots[1]] = weights[pivots[0]];
    else if (res_p1 == '<') weights[pivots[1]] = std::max(1LL, weights[pivots[0]] * FACTOR_LT / 100);
    else weights[pivots[1]] = std::max(1LL, weights[pivots[0]] * FACTOR_GT / 100);

    if (res_p1 == '>' && weights[pivots[1]] == weights[pivots[0]]) {
        weights[pivots[1]] = weights[pivots[0]] + 1;
    }

    // Handle remaining pivots with binary search bracketing
    long long max_bound = BASE_WEIGHT * (N / std::max(1, D) + 10);
    for (int j = 2; j < k_pivots; ++j) {
        int cur = pivots[j], prev = pivots[j-1];
        char res = qm.compare1(cur, prev);

        if (res == '=') {
            weights[cur] = weights[prev];
        } else if (res == '<') {
            权重[当前] = 标准库最大值 (1LL, 权重[前一个] 乘以 FACTOR_LT / 100); } 否则 {
            // Binary search to bracket X_j
            long long X_low = 1, X_high = max_bound;
            bool low_set = false, high_set = false;

            int low_idx = 0, high_idx = j - 2;
            int tries = std::max(1, static_cast<int>(std::ceil(estimate_log2(std::max(1, high_idx - low_idx + 1)))));

            for (int t = 0; t < tries && low_idx <= high_idx && queries_made < Q; ++t) {
                int mid_idx = (low_idx + high_idx) / 2;
                int s = pivots[mid_idx];
                char res_1v2 = qm.compare1v2(cur, prev, s);

                if (res_1v2 == '=') {
                    X_low = X_high = weights[s];
                    low_set = high_set = true;
                    break;
                } else if (res_1v2 == '<') {
                    X_high = weights[s];
                    high_set = true;
                    high_idx = mid_idx - 1;
                } else {
                    X_low = weights[s];
                    low_set = true;
                    low_idx = mid_idx + 1;
                }
            }

            long long est_X;
            if (low_set && !high_set) est_X = X_low * FACTOR_GT / 100;
            else if (!low_set && high_set) est_X = X_high * FACTOR_LT / 100;
            else if (low_set && high_set) est_X = (X_low + X_high) / 2;
            else est_X = weights[prev] * FACTOR_XJ_FALLBACK / 100;

            est_X = std::max(1LL, est_X);
            weights[cur] = weights[prev] + est_X;
        }

        // Ensure monotonicity
        if (weights[cur] < weights[prev]) weights[cur] = weights[prev];
        if (res == '>' && weights[cur] == weights[prev]) weights[cur] = weights[prev] + 1;
    }

    // Estimate non-pivot weights
    std::vector<bool> is_pivot(N, false);
    for (int p : pivots) is_pivot[p] = true;

    for (int i = 0; i < N; ++i) {
        if (is_pivot[i]) continue;

        int low = 0, high = k_pivots - 1, found = -1;
        while (low <= high && queries_made < Q) {
            int mid = (low + high) / 2;
            char res = qm.compare1(i, pivots[mid]);
            if (res == '=') { found = mid; break; }
            else if (res == '<') high = mid - 1;
            else low = mid + 1;
        }
    }
}

```



```

    }

    if (found != -1) {
        weights[i] = weights[pivots[found]];
        continue;
    }

    int pos = low;
    if (pos == 0) {
        long long w0 = weights[pivots[0]];
        if (k_pivots >= 2) {
            long long w1 = weights[pivots[1]];
            if (w1 > w0 && w0 > 0) weights[i] = std::max(1LL, w0 * w0 / w1);
            else weights[i] = std::max(1LL, w0 / 2);
        } else {
            weights[i] = std::max(1LL, w0 / 2);
        }
    } else if (pos == k_pivots) {
        long long wk1 = weights[pivots[k_pivots - 1]];
        if (k_pivots >= 2) {
            long long wk2 = weights[pivots[k_pivots - 2]];
            if (wk1 > wk2 && wk2 > 0) weights[i] = std::max(1LL, wk1 * wk1 / wk2);
            else weights[i] = std::max(1LL, wk1 * 2);
        } else {
            weights[i] = std::max(1LL, wk1 * 2);
        }
    } else {
        long long wl = weights[pivots[pos - 1]];
        long long wr = weights[pivots[pos]];
        if (wl > 0 && wr > 0) {
            weights[i] = static_cast<long long>(std::sqrt(static_cast<double>(wl) * wr));
        } else {
            weights[i] = (wl + wr) / 2;
        }
        weights[i] = std::max(weights[i], wl);
        weights[i] = std::min(weights[i], wr);
    }
    weights[i] = std::max(1LL, weights[i]);
}

// Final 验证
for (int i = 0; i < N; ++i) {
    if (weights[i] <= 0) weights[i] = BASE_WEIGHT;
}

return weights;
}

};

// 分配优化器类 AssignmentOptimizer {
private:

    int N, D;
    std::vector<long long>& weights;
    std::mt19937& rng;

    double calc_variance(const std::vector<long long>& sums, long long total) {
        if (D <= 0) return 1e18;
        double mean = static_cast<double>(total) / D;
        double sum_sq = 0;
        for (long long s : sums) sum_sq += static_cast<double>(s) * s;
        double var = sum_sq / D - mean * mean;
        return std::max(0.0, var);
    }

public:
    AssignmentOptimizer(int N_, int D_, std::vector<long long>& w, std::mt19937& r)
        : N(N_), D(D_), weights(w), rng(r) {}

    std::vector<int> optimize() {
        std::vector<int> assignment(N, 0);
        std::vector<long long> group_sums(D, 0);
        std::vector<std::vector<int>> group_items(D);
        std::vector<int> item_pos(N);

        // Greedy initialization
        std::vector<std::pair<long long, int>> sorted_items;
        for (int i = 0; i < N; ++i) {
            sorted_items.emplace_back(-weights[i], i);
        }
        std::sort(sorted_items.begin(), sorted_items.end());

        long long total_sum = 0;
        for (auto [neg_w, item] : sorted_items) {
            int best_group = 0;
            for (int g = 1; g < D; ++g) {
                if (group_sums[g] < group_sums[best_group]) best_group = g;
            }
            assignment[item] = best_group;
            item_pos[item] = group_items[best_group].size();
            group_items[best_group].push_back(item);
            group_sums[best_group] += weights[item];
            total_sum += weights[item];
        }
    }
}

```

```

double current_var = calc_variance(group_sums, total_sum);

// Enhanced local search with best-of-K
if (D > 1) {
    const int MAX_ITERS = 400;
    const int K_ITEMS = 8;

    for (int iter = 0; iter < MAX_ITERS; ++iter) {
        if ((iter & 31) == 0) {
            auto now = std::chrono::steady_clock::now();
            if (std::chrono::duration_cast<std::chrono::milliseconds>(now - program_start_time) >=
time_limit_ms) break;
        }

        int max_g = 0, min_g = 0;
        for (int g = 1; g < D; ++g) {
            if (group_sums[g] > group_sums[max_g]) max_g = g;
            if (group_sums[g] < group_sums[min_g]) min_g = g;
        }
        if (max_g == min_g || group_items[max_g].empty()) break;

        // Find best relocate from max_g to min_g among top-K heaviest
        std::vector<std::pair<long long, int>> candidates;
        for (int item : group_items[max_g]) {
            candidates.emplace_back(weights[item], item);
        }
        if (candidates.empty()) break;
        std::sort(candidates.begin(), candidates.end(), [](const auto& a, const auto& b) { return a.
first > b.first; });
        // 如果 (整数) 候选者的大小大于 K_ITEMS/candidates.resize(K_ITEMS);

        double best_var = current_var;
        int best_item = -1;
        for (auto [w, item] : candidates) {
            long long new_max = group_sums[max_g] - w;
            long long new_min = group_sums[min_g] + w;
            double new_var = calc_variance({new_max, new_min}, group_sums[max_g] + group_sums[min_g]);
            if (new_var + 1e-12 < best_var) {
                best_var = new_var;
                best_item = item;
            }
        }

        if (best_item == -1) break;

        // Apply move
        long long w = weights[best_item];
        group_sums[max_g] -= w;
        group_sums[min_g] += w;
        current_var = calc_variance(group_sums, total_sum);

        // Update tracking
        int pos = item_pos[best_item];
        int last = group_items[max_g].back();
        if (best_item != last) {
            group_items[max_g][pos] = last;
            item_pos[last] = pos;
        }
        group_items[max_g].pop_back();
        item_pos[best_item] = group_items[min_g].size();
        group_items[min_g].push_back(best_item);
        assignment[best_item] = min_g;
    }

    // Targeted Simulated 退火
    if (D > 1) {
        double T = std::max(1.0, current_var * 0.25);
        double cool_rate = 0.99985;
        std::uniform_real_distribution<double> unif(0.0, 1.0);
        int iterations = 0, no_imp = 0;

        当 (真) { 迭代次数递增; 如果 (迭代次数与 255)
        ==等于 0) {
            auto now = std::chrono::steady_clock::now();
            if (std::chrono::duration_cast<std::chrono::milliseconds>(now - program_start_time) >=
time_limit_ms) break;
            T *= cool_rate;
            if (T < 1e-12) break;
        }

        // Targeted moves: 75% heavy-to-light relocate, 25% swap
        if ((rng() % 4) != 0) {
            // Targeted relocate
            int max_g = 0, min_g = 0;
            for (int g = 1; g < D; ++g) {
                if (group_sums[g] > group_sums[max_g]) max_g = g;
                if (group_sums[g] < group_sums[min_g]) min_g = g;
            }

            如果 (group_items[max_g] 为空) { 递增 no_imp;继续; }

            // 从最大组中挑选重物 (3 个样本中的最佳)

```

```

int item = group_items[max_g][rng() % group_items[max_g].size()];
for (int s = 0; s < 2; ++s) {
    int cand = group_items[max_g][rng() % group_items[max_g].size()];
    if (weights[cand] > weights[item]) item = cand;
}

long long w = weights[item];
long long new_max = group_sums[max_g] - w;
long long new_min = group_sums[min_g] + w;

double new_var = current_var;
new_var -= (static_cast<double>(group_sums[max_g]) * group_sums[max_g]) / D;
new_var -= (static_cast<double>(group_sums[min_g]) * group_sums[min_g]) / D;
new_var += (static_cast<double>(new_max) * new_max) / D;
new_var += (static_cast<double>(new_min) * new_min) / D;

double delta = new_var - current_var;
if (delta < 0 || unif(rng) < std::exp(-delta / T)) {
    // Accept move
    current_var = new_var;
    group_sums[max_g] = new_max;
    group_sums[min_g] = new_min;

    int pos = item_pos[item];
    int last = group_items[max_g].back();
    if (item != last) {
        group_items[max_g][pos] = last;
        item_pos[last] = pos;
    }
    group_items[max_g].pop_back();
    item_pos[item] = group_items[min_g].size();
    group_items[min_g].push_back(item);
    assignment[item] = min_g;

    if (delta < -1e-12) no_imp = 0; else ++no_imp;
} else {
    // Random swap
    int g1 = rng() % D, g2 = rng() % D;
    while (g2 == g1) g2 = rng() % D;
    if (group_items[g1].empty() || group_items[g2].empty()) { ++no_imp; continue; }

    int a = group_items[g1][rng() % group_items[g1].size()];
    int b = group_items[g2][rng() % group_items[g2].size()];
    long long wa = weights[a], wb = weights[b];

    long long new_g1 = group_sums[g1] - wa + wb;
    long long new_g2 = group_sums[g2] - wb + wa;

    double new_var = current_var;
    new_var -= (static_cast<double>(group_sums[g1]) * group_sums[g1]) / D;
    new_var -= (static_cast<double>(group_sums[g2]) * group_sums[g2]) / D;
    new_var += (static_cast<double>(new_g1) * new_g1) / D;
    new_var += (static_cast<double>(new_g2) * new_g2) / D;

    double delta = new_var - current_var;
    if (delta < 0 || unif(rng) < std::exp(-delta / T)) {
        current_var = new_var;
        group_sums[g1] = new_g1;
        group_sums[g2] = new_g2;

        // Swap items
        int pos_a = item_pos[a], pos_b = item_pos[b];
        int back_a = group_items[g1].back(), back_b = group_items[g2].back();
        if (a != back_a) { group_items[g1][pos_a] = back_a; item_pos[back_a] = pos_a; }
        group_items[g1].pop_back();
        if (b != back_b) { group_items[g2][pos_b] = back_b; item_pos[back_b] = pos_b; }
        group_items[g2].pop_back();

        item_pos[b] = group_items[g1].size(); group_items[g1].push_back(b); assignment[b] = g1;
        item_pos[a] = group_items[g2].size(); group_items[g2].push_back(a); assignment[a] = g2;

        if (delta < -1e-12) no_imp = 0; else ++no_imp;
    } else ++no_imp;
}
}

```

如果 (no_imp 大于 N 乘以 12) 则中断; }

返回分配; };

```

int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    program_start_time = std::chrono::steady_clock::now();
    uint64_t seed = std::chrono::duration_cast<std::chrono::nanoseconds>(
        std::chrono::steady_clock::now().time_since_epoch()).count();
    rng_engine.seed(seed);

```

标准输入读取 N_items_global、D_groups_global、Q_total_global;

```
QueryManager qm(N_items_global, Q_total_global, queries_made, rng_engine);
WeightEstimator estimator(qm, N_items_global, D_groups_global, Q_total_global);

std::vector<long long> 权重 = 估计器。estimate_weights();

qm.exhaust_queries();

AssignmentOptimizer optimizer(N_items_global, D_groups_global, weights, rng_engine);
std::vector<int> assignment = optimizer.optimize();

for (int i = 0; i < N_items_global; ++i) {
    std::cout << assignment[i] << (i + 1 == N_items_global ? '\n' : ' ');
}

return 0;
}
// 进化-块-结束
```

清单 5 | ShinkaEvolve 发现的 ahc025 解决方案。

C.4. 专家混合负载均衡损失

```
def load_balancing_loss(
    gate_logits: tuple[torch.Tensor],
    num_experts: int,
    top_k: int = 2,
    attention_mask: Optional[torch.Tensor] = None,
) -> torch.Tensor:
    """ 专家混合模型的负载均衡损失。

    参数
    -----layer_logits:
        list with shape (B, T, total_experts) per layer.
    total_experts:
        专家混合前馈子块内的专家数量。 top_k_experts:
        number of experts chosen per token (k in top-k gating).
    attention_mask:
        可选掩码 (B,T) , 其中 0 标记填充词元。

    返回
    -----torch.Tensor:
        scalar loss to be added to the training objective.
    """
    # determine device & flat token count
    device = gate_logits[0].device
    num_layers = len(gate_logits)
    bsz, seqlen = attention_mask.shape
    n_tokens = bsz * seqlen

    # merge layers into (tokens, layers, experts)
    stacked = torch.stack(gate_logits, dim=-2).to(device)
    logits = stacked.view(n_tokens, num_layers, num_experts)

    # obtain routing information
    _, routing_probs, sel_idx = route_logits_to_scores(logits, top_k)
    sel_mask = F.one_hot(sel_idx, num_experts)

    if attention_mask is None:
        # average over all tokens
        avg_sel = sel_mask.float().mean(dim=0)
        avg_prob = routing_probs.mean(dim=0)
    else:
        # expand & apply mask
        m_exp = (
            attention_mask.unsqueeze(-1)
            .unsqueeze(-1)
            .unsqueeze(-1)
            .expand(bsz, seqlen, num_layers, top_k, num_experts)
            .reshape(-1, num_layers, top_k, num_experts)
        )
        avg_sel = sel_mask.float().mul(m_exp).sum(dim=0) / m_exp.sum(dim=0)

        p_mask = (
            attention_mask.unsqueeze(-1)
            .unsqueeze(-1)
            .expand(bsz, seqlen, num_layers, num_experts)
            .reshape(-1, num_layers, num_experts)
        )
        avg_prob = routing_probs.mul(p_mask).sum(dim=0) / p_mask.sum(dim=0)

    # mismatch penalty
    per_layer = avg_sel * avg_prob.unsqueeze(-2)
    main_loss = per_layer.mean(0).sum() * num_experts

    # --- Minimum usage regularizer: softly penalize underused experts ---
    # avg_sel: (layers, top_k, experts)
    # For each expert, sum over top_k to get total selection per expert per layer
    avg_sel_sum = avg_sel.sum(dim=-2) # (layers, experts)
    # Normalize so that sum over experts = 1 per layer
    avg_sel_norm = avg_sel_sum / (avg_sel_sum.sum(dim=-1, keepdim=True) + 1e-8)

    # Compute entropy of avg_prob per layer (routing distribution)
    entropy = -(avg_prob * torch.log(avg_prob + 1e-8)).sum(dim=-1) # (layers,)
    max_entropy = torch.log(torch.tensor(num_experts, dtype=avg_prob.dtype, device=avg_prob.device))
    entropy_scale = 1.5 - entropy / (max_entropy + 1e-8) # ranges from 0.5 (uniform) to 1.5 (concentrated)

    # Penalty: encourage each expert to be used at least min_threshold
    min_threshold = 0.01 * (64.0 / num_experts)

    min_usage_penalty = torch.relu(min_threshold - avg_sel_norm).sum(dim=-1) # (layers,)
    penalty_coeff = 0.1

    # Final loss: main + entropy-scaled min usage penalty
    return main_loss + penalty_coeff * (min_usage_penalty * entropy_scale).mean()
```

清单 6 | ShinkaEvolve 发现的专家混合负载均衡损失。